Contents

- 1. Principles
- 2. Conventions
  - 2.1 Primitive types at the boundary
  - 2.2 Naming
  - 2.3 Export shape
  - 2.4 Synchronous vs asynchronous calls
  - 2.5 Versioning and ABI stability
  - 2.6 Threading, reentrancy, errors
- 3. Objects and handles
  - 3.1 Handle value
  - 3.2 Object types
  - 3.3 Rights
  - 3.4 Signals
  - 3.5 Generic handle and object calls
- 4. Errors
- 5. Ports and completions
  - 5.1 Calls
  - 5.2 Completion packet
  - 5.3 Backend note
- 6. Memory
  - 6.1 Model
  - 6.2 Types
  - 6.3 Calls (all synchronous)
  - 6.4 Backend
- 7. Threads and synchronisation
  - 7.1 Model
  - 7.2 Types
  - 7.3 Calls (all synchronous)
  - 7.4 Wait/wake on address
  - 7.5 Backend
- 8. Time
  - 8.1 Clocks
  - 8.2 Timers
- 9. Events
- 10. Channels
  - 10.1 Model
  - 10.2 Calls (all synchronous, non-blocking; readiness via Object.waitAsync)
  - 10.3 Tessera IPC wire format v1

    - 10.4 Backend
- 11. Processes and jobs
    - 11.1 Model
    - 11.2 Types
    - 11.3 Calls
    - 11.4 The bootstrap protocol
    - 11.5 Runtime initialisation sequence (no libc)
    - 11.6 Entitlements
    - 11.7 Backend
- 12. The System channel protocol (tessera.system)
- 13. Namespace and files
    - 13.1 Model
    - 13.2 Path rules
    - 13.3 Types
    - 13.4 Namespace calls (asynchronous, kind Namespace)
    - 13.5 File calls (data path)
    - 13.6 Implementation: the Namespace service
    - 13.7 Enforcement ledger for this domain
- 14. Grants, presence and pickers
    - 14.1 Model
    - 14.2 Types
    - 14.3 Calls (asynchronous, kind Grant, via tessera.presence)
    - 14.4 Other interactive authority
    - 14.5 The child-brokering flow, end to end
    - 14.6 Backend
- 15. Network
    - 15.1 Model
    - 15.2 Types
    - 15.3 Calls
    - 15.4 Backend
- 16. Device streams
    - 16.1 Model
    - 16.2 Types
    - 16.3 Calls
    - 16.4 Audio specifics
    - 16.5 Backend
- 17. GPU and fences
    - 17.1 Model
    - 17.2 Types
    - 17.3 Calls
    - 17.4 Display handoff (for reference; protocol owned by MosaicDisplay)
    - 17.5 Backend

- 18. Power and thermal
  - 18.1 Types
  - 18.2 Calls
- 19. Random
- 20. Diagnostics and system information
  - 20.1 Logging and tracing
  - 20.2 Crash and debug (debug image / entitled only)
  - 20.3 System information
- 21. Kernel backends
  - 21.1 Linux backend mapping
  - 21.2 Linux app seccomp allowlist (generated; base set)
  - 21.3 XNU backend mapping
  - 21.4 Enforcement ledger
  - 21.5 The kernel-specific residue
- 22. Consumers
  - 22.1 Tessera Rust std
  - 22.2 A Tessera C std
  - 22.3 The Kira runtime and frameworks
- 23. Call index
- 24. Decisions still open (for Priam)

LibTessera — System Interface Specification

Version 0.1 · target CalVer 2026.10 · status: design draft for review

LibTessera is the sole system interface of TesseraOS. It is written in Kira, exported to C through `@Export`, and linked by the Tessera Rust std, a future Tessera C std, every system service, and the Kira runtime. It is the surface of a full operating system. The kernel underneath — Linux today, XNU or a native kernel later — is an implementation detail that no consumer can observe through this API.

Reading guide: §1–5 are the ground rules every domain obeys. §6–20 are the domains, each with its objects, structs, calls, rights, errors and backend notes. §21 is the kernel backends and the enforcement ledger. §22 maps consumers (Rust std, C std, Kira runtime) onto the calls. §23 is the complete call index. §24 lists the decisions still open.

---

1. Principles

1. **Everything is a handle.** Every kernel or system resource is an object reached through a process-local, unforgeable, typed handle carrying rights. There are no integer file descriptors, no PIDs used as authority, no global names.
2. **Rights, not identity.** What a process may do is what its handles allow plus what its entitlement set unlocks. Nothing checks "who are you" at call time; the check happened when the handle was minted.
3. **Completion, not blocking.** Any operation that leaves the process (disk, network, a service, hardware) is asynchronous: it is submitted with a `(port, key)` and produces exactly one `Completion` packet. Synchronous consumers wait on a port.
4. **No ambient authority.** A process starts with exactly the handles listed in its `Startup` message. There is no working directory, no environment variables, no inherited descriptor table, no root namespace.
5. **Not POSIX.** No libc, no `errno`, no signals, no `fork`, no absolute paths, no `select`. Tessera-native code never touches any of these; the POSIX tier is a separate, notarized, time-limited world.

6. **Kernel-agnostic by construction.** No call, struct, flag or error names a kernel concept. Each object has a semantic definition independent of any backend, and the backend section proves each backend can implement it.
7. **Fixed-width, explicit, little-endian.** Kira's `Number` never crosses this boundary, nor do platform-sized integers. Every field has a stated width, offset and meaning.
8. **Explainable failure.** Every error is a structured value: code, subsystem, and a detail word that for policy failures is an audit reference the user can be shown.
9. **Additive evolution.** Declarations carry CalVer availability (`@introduced`, `@deprecated`). Nothing is ever removed or reshaped; new behaviour is a new call or a flag whose zero value preserves old behaviour.
10. **Mechanism here, policy in services.** LibTessera never shows UI, never decides policy. Entitlements, grants, pickers and prompts live in system services reached through channels; LibTessera provides the objects and the transport.

---

2. Conventions

2.1 Primitive types at the boundary

Kira	C	Width	Meaning
`UInt8/16/32/64`, `Int32/64`	`uint8_t` … `int64_t`	as named	plain integers
`Bool`	`uint8_t`	1	0 = false, 1 = true, other values are `InvalidArgument`
`Address`	`uint64_t`	8	virtual address in the calling process
`Size`	`uint64_t`	8	byte count
`Instant`	`uint64_t`	8	nanoseconds on a stated clock
`Duration`	`uint64_t`	8	nanoseconds
`Deadline`	`uint64_t`	8	an `Instant` on `Clock.Monotonic`; `0` = poll (do not block); `UInt64.max` = `INFINITE`

Kira	C	Width	Meaning
`Handle<T>`	`tessera_handle_t` (`uint64_t`)	8	typed in Kira, untyped in C; `0` = `INVALID`
`Key`	`uint64_t`	8	caller-chosen correlation value for completions
`Rights`	`uint64_t`	8	bitset, §3.3
`Signals`	`uint32_t`	4	bitset, §3.4
`Buffer`	`tessera_buffer_t`	16	`{ address: Address, length: Size }`
`Error`	`tessera_error_t`	16	§4
`Completion`	`tessera_completion_t`	64	§5.2

Arrays cross as `(pointer, count)` pairs. Strings are UTF-8 bytes in a `Buffer`, never NUL-terminated, never longer than the stated limit. Enums cross as `UInt32` unless stated. All structs use natural C alignment with explicit padding fields named `reserved`, which callers must zero and readers must ignore.

2.2 Naming

- Kira: `Tessera.<Domain>.<verb>`, types `PascalCase`, enum cases `PascalCase`. This document writes calls as `Domain.verb`.
- C: `tessera_<domain>_<verb>`, types `tessera_<name>_t`, constants `TESSERA_<DOMAIN>_<NAME>`.
- Verb vocabulary: `create` (new object from nothing), `open` (handle to an existing resource), `close`, `read`/`write`, `get`/`set`, `wait`, `signal`, `cancel`, `list`, `lookup`, `info`.

2.3 Export shape

Every public declaration carries `@Export("tessera_…")` and `@introduced("YYYY.MM")`. The mapping from Kira to C is mechanical:

- `-> Result<Void, Error>` becomes `tessera_error_t f(...)`.
- `-> Result<T, Error>` becomes `tessera_error_t f(..., T* out)`; multiple results become multiple trailing out-pointers in declaration order.
- `Handle<T>` becomes `tessera_handle_t`; `Handle<T>?` (optional) is `0` when absent.
- Enums with payloads never cross; boundary enums are `UInt32`-backed.

- Structs at the boundary are declared with `@Export` layout: field order is layout order, no reordering, explicit padding.
- `noreturn` functions (`Process.exit`, `Thread.exit`) are marked `_Noreturn` in the generated header.

The generated header is `tessera.h`. The Rust std links `libtessera` and re-declares these prototypes; it never sees Kira.

2.4 Synchronous vs asynchronous calls

A call is **synchronous** if it never leaves the process and never waits on hardware: handle-table operations, memory mapping, clock reads, port/channel/event operations, thread control, wait/wake on address (bounded by a deadline the caller chose). Synchronous calls return `Result<T, Error>` and are complete when they return.

A call is **asynchronous** if it touches disk, the network, a device, or a system service. Asynchronous calls take `port: Handle<Port>, key: Key` as their last two parameters, return `Result<Void, Error>` for the *submission only* (validation errors, missing rights), and then deliver **exactly one** `Completion` to `port` carrying `key` — success, failure, or `Cancelled`. Calls documented as *multi-shot* deliver repeated completions until cancelled or the object is closed, and one final completion with `Cancelled`.

Rules for in-flight operations:

- Memory referenced by a submitted operation (buffers, address structs) must stay valid and unmodified until its completion is observed.
- Closing the object handle cancels its in-flight operations; they complete with `Cancelled`.
- Closing the port drops all pending completions; in-flight operations still run to completion but their results are discarded.
- `Port.cancel(port, key)` cancels every in-flight operation on that port with that key.

Thread-port convention. Synchronous consumers (Rust std, C std) lazily create one `Port` per thread, submit with `key = 0`, and call `Port.wait(port, 1, INFINITE)`. That is the entire cost of "sync" — there is no synchronous mode in LibTessera.

2.5 Versioning and ABI stability

- Availability is per declaration: `@introduced("2026.10")`, `@deprecated("2029.01", reason:)`. Deprecated symbols keep working for at least four years after deprecation, then move to a compatibility shim that still resolves.
- No symbol is ever renamed, removed, or changes signature. A changed shape is a new symbol; the old one stays.
- Info structs are versioned by *topic*: `Object.info(handle, topic, version, out)`; a new field means a new topic version, and older versions keep being served.
- Options and flags: the zero value of every flag is the original behaviour; new flags are appended.
- `System.info(.Version)` reports both the OS CalVer and the LibTessera ABI CalVer so consumers can gate on either.

2.6 Threading, reentrancy, errors

- Every call is thread-safe. Handles may be used from any thread of the owning process concurrently.
- Calls never allocate on the caller's behalf except where a returned handle is documented; callers own every buffer.
- Calls never call back into the caller. There are no callbacks anywhere in the ABI.
- No call is ever "interrupted": there are no signals. `Interrupted` is not an error code.

---

3. Objects and handles

3.1 Handle value

A handle is a `UInt64`. The value `0` is `INVALID` and is never a valid handle. Non-zero values are opaque; internally they encode a table index and a generation counter so a closed handle cannot be confused with a later one. Handles are **process-local names**: the same object seen by two processes has unrelated values. The maximum number of open handles per process is reported by `System.info(.Limits)` (default 65 536).

3.2 Object types

```
enum ObjectType: UInt32
  None        = 0
  Port        = 1    completion queue                                    (§5)
  Event       = 2    signalable object                                   (§9)
  EventPair   = 3    two linked events with peer-death                   (§9)
  Timer       = 4    clock-driven completion source                      (§8)
  Channel     = 5    bidirectional message + handle transport            (§10)
  Memory      = 6    pageable memory object, mappable                    (§6)
  Thread      = 7    thread of execution                                 (§7)
  Process     = 8    address space + handle table + threads              (§11)
  Job         = 9    tree node owning processes and child jobs           (§11)
  Node        = 10   namespace capability: a file or directory           (§13)
  File        = 11   data handle for a file's bytes                      (§13)
  Watch       = 12   change-notification stream on a Node                (§13)
  Endpoint    = 13   network endpoint                                    (§15)
  Stream      = 14   device data stream (sensor, audio, camera, haptic) (§16)
  GpuDevice   = 15   GPU device                                           (§17)
  Fence       = 16   binary or timeline synchronisation object           (§17)
  WakeLock    = 17   power management lock                               (§18)
  Presence    = 18   the app's right to interact with the user           (§14)
  // 19–255 reserved for Tessera; 256+ reserved for future backends
```

3.3 Rights

`Rights` is a 64-bit set. Bits 0–15 are generic and have the same meaning on every object type; bits 16–31 are type-specific; bits 32–63 are reserved. `RIGHTS_SAME` (`UInt64.max`) means "the same rights as the source" and is only valid in `Handle.duplicate`.

```
Duplicate   = 1 << 0    may create another handle to the object
Transfer    = 1 << 1    may be sent through a channel
Read        = 1 << 2    read bytes / receive / observe state
Write       = 1 << 3    write bytes / send / mutate state    (fs: Update)
Map         = 1 << 4    may be mapped into the address space
Execute     = 1 << 5    may be mapped executable
Signal      = 1 << 6    may assert user signals
Wait        = 1 << 7    may wait on signals / receive completions
Inspect     = 1 << 8    may read info topics
Manage      = 1 << 9    may destroy, kill, resize, configure, set policy
SetProperty = 1 << 10   may set name / attributes
Enumerate   = 1 << 11   fs: List
Create      = 1 << 12   fs: Create
Delete      = 1 << 13   fs: Delete
```

```
Traverse    = 1 << 14   fs: lookup into child directories
Grant       = 1 << 15   Presence: may request grants and present pickers
```

The five filesystem capabilities from the permission model map exactly: **Read** → **`Read`**, **List** → **`Enumerate`**, **Create** → **`Create`**, **Update** → **`Write`**, **Delete** → **`Delete`**, plus `Traverse` for entering subdirectories. They are independent bits: `Create` does not imply `Write` or `Delete`; `Read` does not imply `Enumerate`.

Default rights when an object is created by the calling process are `Duplicate | Transfer | Read | Write | Signal | Wait | Inspect | Manage | SetProperty` plus the type's natural extras (`Map` for Memory, `Enumerate | Create | Delete | Traverse` on a Node the process is entitled to). Handles minted by a service carry exactly what the grant allows. `Presence` handles never carry `Transfer` or `Duplicate`.

3.4 Signals

`Signals` is a 32-bit set. Bits 0–15 are defined per object type, bits 16–23 reserved, bits 24–31 are **user signals** (`USER_0 … USER_7`) settable on `Event` and `EventPair` only.

```
READABLE    = 1 << 0    something to read / a completion available
WRITABLE    = 1 << 1    can accept a write without ShouldWait
PEER_CLOSED = 1 << 2    the other end of a pair is gone
SIGNALED    = 1 << 3    Event/Fence/Timer fired
TERMINATED  = 1 << 4    Thread/Process/Job ended
ERROR       = 1 << 5    object is in an error state (Endpoint, Stream)
OVERRUN     = 1 << 6    Stream: data lost
```

Per type: Channel `READABLE | WRITABLE | PEER_CLOSED`; Endpoint `READABLE | WRITABLE | PEER_CLOSED | ERROR`; Thread/Process/Job `TERMINATED`; Event/EventPair `SIGNALED | PEER_CLOSED | USER_*`; Fence `SIGNALED`; Watch `READABLE | PEER_CLOSED`; Stream `READABLE | WRITABLE | ERROR | OVERRUN`; Port `READABLE`.

3.5 Generic handle and object calls

All synchronous.

```
Handle.close(handle: Handle<Any>) -> Result<Void, Error>
C  tessera_error_t tessera_handle_close(tessera_handle_t handle);
```

Invalidates the handle. In-flight operations on it complete with `Cancelled`. The object is destroyed when its last handle in every process is closed (and, for Process/Thread/Job, when it has terminated). Errors: `InvalidHandle`.

```
Handle.closeMany(handles: [Handle<Any>], count: UInt32) -> Result<Void, Error>
C  tessera_error_t tessera_handle_close_many(const tessera_handle_t* handles,
uint32_t count);
```

Closes all listed handles; continues past invalid ones and reports `InvalidHandle` if any were.

```
Handle.duplicate(handle: Handle<T>, rights: Rights) -> Result<Handle<T>, Error>
C  tessera_error_t tessera_handle_duplicate(tessera_handle_t handle, uint64_t
rights, tessera_handle_t* out);
```

Requires `Duplicate`. `rights` must be a subset of the source's rights or `RIGHTS_SAME`. Errors: `InvalidHandle`, `AccessDenied` (missing Duplicate or superset requested).

```
Handle.replace(handle: Handle<T>, rights: Rights) -> Result<Handle<T>, Error>
C  tessera_error_t tessera_handle_replace(tessera_handle_t handle, uint64_t
rights, tessera_handle_t* out);
```

Atomically returns a new handle with the (reduced) rights and invalidates the old one. Does not require `Duplicate` — this is how a process narrows its own authority before passing a handle on.

```
Handle.info(handle: Handle<Any>) -> Result<HandleInfo, Error>
C  tessera_error_t tessera_handle_info(tessera_handle_t handle,
tessera_handle_info_t* out);

struct HandleInfo (32 bytes)
  type:      ObjectType (u32)   @0
  reserved0: u32                @4
  rights:    Rights (u64)       @8
  objectId:  u64                @16   stable within the process's lifetime;
equal for handles to the same object
  reserved1: u64                @24
```

```
Object.info(handle: Handle<Any>, topic: InfoTopic, version: UInt32, out:
Buffer) -> Result<Size, Error>
```

```
C  tessera_error_t tessera_object_info(tessera_handle_t handle, uint32_t topic,
uint32_t version, tessera_buffer_t out, uint64_t* out_written);
```

Requires `Inspect`. Writes the topic struct for the requested version. Returns `BufferTooSmall` with the required size in `Error.detail`. Topics are listed per object type in their sections (`ProcessInfo`, `NodeInfo`, `EndpointInfo`, …).

```
Object.setName(handle: Handle<Any>, name: Buffer) -> Result<Void, Error>
C  tessera_error_t tessera_object_set_name(tessera_handle_t handle,
tessera_buffer_t name);
```

Requires `SetProperty`. Diagnostic name, ≤ 64 bytes UTF-8, shown in tooling. Errors: `InvalidArgument`.

```
Object.waitAsync(handle: Handle<Any>, signals: Signals, options: WaitOptions,
port: Handle<Port>, key: Key) -> Result<Void, Error>
C  tessera_error_t tessera_object_wait_async(tessera_handle_t handle, uint32_t
signals, uint32_t options, tessera_handle_t port, uint64_t key);

enum WaitOptions: UInt32   Once = 0 (default), Repeat = 1, Edge = 2
```

Requires `Wait`. Posts a `Completion` of kind `Signal` when any of `signals` is asserted; `result0` = signals asserted at delivery. `Once` delivers one packet and disarms; `Repeat` re-arms until cancelled (multi-shot); `Edge` only reports transitions. This is the universal readiness mechanism — the equivalent of every `poll` / `epoll` / `kqueue` use, expressed as completions on the same port as everything else.

```
Object.signal(handle: Handle<Event|EventPair>, set: Signals, clear: Signals) ->
Result<Void, Error>
C  tessera_error_t tessera_object_signal(tessera_handle_t handle, uint32_t set,
uint32_t clear);
```

Requires `Signal`. Only user bits and `SIGNALED` may be set/cleared. Errors: `AccessDenied`, `WrongType`, `InvalidArgument`.

---

4. Errors

```
struct Error (16 bytes)
  code:      ErrorCode (u32)   @0    0 = Ok
  subsystem: Subsystem (u16)   @4    which layer produced it
  flags:     u16               @6    bit0 HAS_AUDIT_REF, bit1 HAS_BACKEND_CODE,
bit2 RETRYABLE
  detail:    u64               @8    audit reference, required size, backend
code, or 0
```

C: `tessera_error_t`, returned by value from every function; `TESSERA_OK` is the all-zero value. `tessera_error_ok(e)` is `e.code == 0`.

```
enum Subsystem: UInt16
  LibTessera = 0, Kernel = 1, Namespace = 2, Presence = 3, Launcher = 4,
Devices = 5,
  Network = 6, Display = 7, Power = 8, Audit = 9, Gpu = 10
```

```
enum ErrorCode: UInt32
  Ok                  = 0
  // argument / handle
  InvalidArgument     = 1     a parameter is malformed
  InvalidHandle       = 2     handle is 0, closed, or not owned by this process
  WrongType           = 3     handle is not the required object type
  AccessDenied        = 4     handle lacks the required right
  NotEntitled         = 5     process lacks the required entitlement (detail =
audit ref)
  OutOfRange          = 6     offset/length/index beyond the object
  BufferTooSmall      = 7     detail = required size
  NameTooLong         = 8
  InvalidPath         = 9     path violates §13.2 (detail = byte offset of the
violation)
  Unsupported         = 10    valid request the backend/device cannot honour
  // state
  NotFound            = 20
  AlreadyExists       = 21
  Busy                = 22    resource in use (e.g. locked, mapped, streaming)
  BadState            = 23    object not in the state the call requires
  ShouldWait          = 24    non-blocking call would block; wait for the
relevant signal
  Timeout             = 25    deadline passed
  Cancelled           = 26    operation cancelled
  PeerClosed          = 27    other end of channel/eventpair/endpoint gone
  // resources
  OutOfMemory         = 30
  NoSpace             = 31    storage exhausted
```

```
  QuotaExceeded       = 32    job policy limit hit (detail = which limit)
  TooManyHandles      = 33
  // namespace
  NotDirectory        = 40
  IsDirectory         = 41
  NotEmpty            = 42
  ProtectedNode       = 43    operation forbidden on a protected root (detail =
audit ref)
  GrantExpired        = 44    detail = audit ref
  GrantRevoked        = 45    detail = audit ref
  // presence
  NoInteraction       = 50    InteractionToken missing, spent, or expired
  NotFrontmost        = 51    requesting surface is not front-facing
  NotMainProcess      = 52    caller does not hold Presence
  UserDenied          = 53    the user said no (detail = audit ref)
  // network
  ConnectionRefused   = 60
  ConnectionReset     = 61
  Unreachable         = 62
  AddressInUse        = 63
  AddressNotAvailable = 64
  // devices / gpu
  DeviceLost          = 70
  DeviceBusy          = 71
  FormatUnsupported   = 72
  Overrun             = 73    stream data lost
  Underrun            = 74    stream starved
  // catch-all
  Internal            = 90    backend failure; detail = backend code
(flags.HAS_BACKEND_CODE)
```

Policy failures (`NotEntitled`, `UserDenied`, `ProtectedNode`, `GrantExpired`, `GrantRevoked`) always carry an audit reference: a 64-bit id into the central grant/request log kept by the Audit service, so the failure can be explained in Settings instead of collapsing into an opaque error.

---

5. Ports and completions

A `Port` is a completion queue. It is the only way asynchronous results, signal notifications, timer expirations, and user-posted packets reach a process. A port has no owner thread; any thread may wait on it, and multiple threads waiting on one port form a work pool.

5.1 Calls

```
Port.create(options: PortOptions) -> Result<Handle<Port>, Error>
C  tessera_error_t tessera_port_create(uint32_t options, tessera_handle_t*
out);
enum PortOptions: UInt32   None = 0
```

```
Port.wait(port: Handle<Port>, out: [Completion], max: UInt32, deadline:
Deadline) -> Result<UInt32, Error>
C  tessera_error_t tessera_port_wait(tessera_handle_t port,
tessera_completion_t* out, uint32_t max, uint64_t deadline, uint32_t*
out_count);
```

Requires `Wait`. Dequeues up to `max` completions, blocking until at least one is available or `deadline` passes. Returns `Timeout` with count 0 when the deadline passes; `deadline = 0` polls. Completions from one object are delivered in submission order; across objects order is by completion time.

```
Port.post(port: Handle<Port>, packet: Completion) -> Result<Void, Error>
C  tessera_error_t tessera_port_post(tessera_handle_t port, const
tessera_completion_t* packet);
```

Requires `Write`. Enqueues a user packet (`kind` is forced to `User`). Returns `ShouldWait` if the port holds more than its capacity of pending user packets (default 65 536) — user packets are bounded so a process cannot exhaust memory through its own port.

```
Port.cancel(port: Handle<Port>, key: Key) -> Result<UInt32, Error>
C  tessera_error_t tessera_port_cancel(tessera_handle_t port, uint64_t key,
uint32_t* out_cancelled);
```

Cancels every in-flight asynchronous operation and armed `waitAsync` targeting this port with this key. Each cancelled operation still delivers its completion, with `Cancelled`. Returns how many were cancelled.

5.2 Completion packet

```
struct Completion (64 bytes)
  key:       u64                @0    the submitter's Key
  error:     Error (16)         @8    Ok, or why the operation failed
  kind:      CompletionKind u32 @24   what kind of operation completed
  flags:     u32                @28   bit0 MORE (multi-shot: further packets
```

```
will follow)
  result0:   u64                @32   primary result (bytes transferred, new
handle, signals, …)
  result1:   u64                @40   secondary result (cursor, timestamp, …)
  aux:       u64                @48   tertiary (handle, length, …)
  timestamp: Instant            @56   Clock.Monotonic at completion
```

```
enum CompletionKind: UInt32
  User = 0, Signal = 1, Timer = 2, Io = 3, Namespace = 4, Launch = 5, Grant =
6,
  Net = 7, Stream = 8, Gpu = 9, Power = 10, Service = 11
```

Every asynchronous call documents what it places in `result0/result1/aux`. A returned handle is always in `result0` and is already in the caller's handle table when the packet is observed; if the packet's `error` is not `Ok`, `result0` is `0` and nothing was added.

5.3 Backend note

Linux: one `io_uring` instance per Port (`IORING_SETUP_SINGLE_ISSUER` off, since any thread may submit; `IORING_SETUP_DEFER_TASKRUN` off), with `waitAsync` as `POLL_ADD`, timers as `TIMEOUT`, cancellation as `ASYNC_CANCEL` by user-data, and service-mediated operations completed through an internal eventfd-backed side queue merged in `Port.wait`. XNU: one `kqueue` per Port; asynchronous file/network I/O is done by an internal worker pool with results posted through `EVFILT_USER`. Native kernel: a real completion queue object.

---

6. Memory

6.1 Model

A `Memory` object is a resizable, pageable, reference-counted container of bytes — the only kind of memory a process can map. Anonymous memory, shared memory, file data (through a `File` handle), device buffers and GPU allocations are all `Memory` or mappable like it. The address space itself is not an object; mappings are created and destroyed by address.

Page size is a system constant reported by `System.info(.Machine).pageSize` — 16 KiB on ARM64, 4 KiB on the x64 compromise tier. All offsets and lengths in this section are page-aligned unless stated.

6.2 Types

```
enum MemoryOptions: UInt32 (flags)
  None            = 0
  Resizable       = 1 << 0    Memory.resize allowed
  Contiguous      = 1 << 1    physically contiguous (entitlement
tessera.driver)
  DeviceShareable = 1 << 2    importable by devices/GPU/compositor (dma-buf /
IOSurface class)
  Discardable     = 1 << 3    system may reclaim contents under pressure; see
Memory.lock
  NoCommit        = 1 << 4    reserve only; pages allocated on first touch
(default is also lazy; this forbids pre-commit)

enum Protection: UInt32 (flags)
  None = 0, Read = 1 << 0, Write = 1 << 1, Execute = 1 << 2
  // Write|Execute together requires entitlement tessera.jit and the Execute
right

enum MapOptions: UInt32 (flags)
  None        = 0
  Fixed       = 1 << 0    map exactly at hint; fail with AlreadyExists if
occupied (never replaces)
  Private     = 1 << 1    copy-on-write private view (writes never reach the
object)
  GuardBelow  = 1 << 2    add one inaccessible guard page below the mapping
  GuardAbove  = 1 << 3    add one inaccessible guard page above
  HugePages   = 1 << 4    prefer large pages (advisory)

enum CachePolicy: UInt32   Cached = 0, WriteCombining = 1, Uncached = 2
(device memory only)

struct MemoryInfo (topic Memory, v1, 48 bytes)
  size:        u64 @0    current size in bytes
  committed:   u64 @8    bytes currently backed by physical pages
  options:     u32 @16
  cachePolicy: u32 @20
  objectId:    u64 @24
  parentId:    u64 @32   0 unless a slice
  reserved:    u64 @40

struct MappingInfo (56 bytes)   result of Memory.query
  base:        Address @0
  length:      Size    @8
```

```
  protection:  u32     @16
  options:     u32     @20
  objectId:    u64     @24   0 for reserved-only ranges
  objectOffset:u64     @32
  committed:   u64     @40
  reserved:    u64     @48
```

6.3 Calls (all synchronous)

```
Memory.create(size: Size, options: MemoryOptions) -> Result<Handle<Memory>,
Error>
C  tessera_error_t tessera_memory_create(uint64_t size, uint32_t options,
tessera_handle_t* out);
```

Zero-filled. Rights: default + `Map`. Errors: `InvalidArgument` (size 0 or unaligned), `OutOfMemory`, `QuotaExceeded`, `NotEntitled` (Contiguous).

```
Memory.resize(memory: Handle<Memory>, size: Size) -> Result<Void, Error>
C  tessera_error_t tessera_memory_resize(tessera_handle_t memory, uint64_t
size);
```

Requires `Manage`, object created `Resizable`. Growing zero-fills; shrinking discards. Fails `Busy` if any mapping covers the range being removed.

```
Memory.slice(parent: Handle<Memory>, offset: Size, length: Size, options:
SliceOptions) -> Result<Handle<Memory>, Error>
C  tessera_error_t tessera_memory_slice(tessera_handle_t parent, uint64_t
offset, uint64_t length, uint32_t options, tessera_handle_t* out);
enum SliceOptions: UInt32   Shared = 0 (window onto parent), CopyOnWrite = 1
(snapshot, lazily copied)
```

Requires `Duplicate`. A slice handle can be given away with narrower rights, which is how a process shares one region of a buffer.

```
Memory.read(memory: Handle<Memory>, offset: Size, out: Buffer) -> Result<Void,
Error>
Memory.write(memory: Handle<Memory>, offset: Size, in: Buffer) -> Result<Void,
Error>
C  tessera_error_t tessera_memory_read(tessera_handle_t memory, uint64_t
offset, tessera_buffer_t out);
C  tessera_error_t tessera_memory_write(tessera_handle_t memory, uint64_t
offset, tessera_buffer_t in);
```

Copy without mapping. Requires `Read` / `Write`. Byte-granular. Errors: `OutOfRange`.

```
Memory.map(source: Handle<Memory|File>, offset: Size, length: Size, protection:
Protection, options: MapOptions, hint: Address) -> Result<Address, Error>
C  tessera_error_t tessera_memory_map(tessera_handle_t source, uint64_t offset,
uint64_t length, uint32_t protection, uint32_t options, uint64_t hint,
uint64_t* out_address);
```

Requires `Map`, plus `Read` for `Protection.Read`, `Write` for `Protection.Write` (unless `Private`), `Execute` for `Protection.Execute`. `source` may be a `File` (file-backed mapping; `Write` reaches the file unless `Private`). Errors: `AccessDenied`, `NotEntitled` (W|X), `OutOfRange`, `OutOfMemory`, `AlreadyExists` (Fixed).

```
Memory.unmap(base: Address, length: Size) -> Result<Void, Error>
C  tessera_error_t tessera_memory_unmap(uint64_t base, uint64_t length);
```

Unmaps every page in range; partial unmap of a mapping splits it. Unmapping a reserved range releases it.

```
Memory.protect(base: Address, length: Size, protection: Protection) ->
Result<Void, Error>
C  tessera_error_t tessera_memory_protect(uint64_t base, uint64_t length,
uint32_t protection);
```

Subject to the same right checks as `map` against each underlying object. W|X requires `tessera.jit`.

```
Memory.commit(base: Address, length: Size) -> Result<Void, Error>
Memory.decommit(base: Address, length: Size) -> Result<Void, Error>
C  tessera_error_t tessera_memory_commit(uint64_t base, uint64_t length);
C  tessera_error_t tessera_memory_decommit(uint64_t base, uint64_t length);
```

`commit` guarantees physical backing now (fails `OutOfMemory` instead of faulting later). `decommit` releases backing; the pages read as zero afterwards. This is the allocator's `madvise` equivalent and the only one.

```
Memory.reserve(length: Size, alignment: Size, hint: Address) -> Result<Address,
Error>
C  tessera_error_t tessera_memory_reserve(uint64_t length, uint64_t alignment,
uint64_t hint, uint64_t* out_base);
```

Reserves an inaccessible address range that later `map(.Fixed)` calls can fill. Allocators (the Kira runtime allocator, Rust's) build arenas from this.

```
Memory.query(address: Address) -> Result<MappingInfo, Error>
C  tessera_error_t tessera_memory_query(uint64_t address,
tessera_mapping_info_t* out);
```

Describes the mapping or reserved range containing `address`. `NotFound` if unmapped.

```
Memory.lock(memory: Handle<Memory>) -> Result<Bool, Error>
Memory.unlock(memory: Handle<Memory>) -> Result<Void, Error>
C  tessera_error_t tessera_memory_lock(tessera_handle_t memory, uint8_t*
out_contents_intact);
C  tessera_error_t tessera_memory_unlock(tessera_handle_t memory);
```

For `Discardable` objects: `lock` pins the contents and reports whether they survived (false = the system discarded them; the object reads as zero). Caches use this.

```
Memory.setCachePolicy(memory: Handle<Memory>, policy: CachePolicy) ->
Result<Void, Error>
C  tessera_error_t tessera_memory_set_cache_policy(tessera_handle_t memory,
uint32_t policy);
```

Requires `Manage` and `tessera.driver`. Only before the first mapping.

6.4 Backend

Linux: `Memory` = `memfd_create` (sealed for size where not `Resizable`; `MFD_HUGETLB` for `HugePages`), `DeviceShareable` = `udmabuf` or a GPU allocation exported as dma-buf, `Contiguous` = CMA via the Devices service. `map` = `mmap` with `MAP_FIXED_NOREPLACE` for `Fixed`, `MAP_PRIVATE` for `Private`; guards are `PROT_NONE` pages; `reserve` = `mmap(PROT_NONE, MAP_NORESERVE)`; `commit` = `MADV_POPULATE_WRITE`; `decommit` = `MADV_DONTNEED` on shared / `MADV_FREE` on private; `Discardable` = `MADV_FREE` semantics tracked by the library. XNU: `mach_make_memory_entry_64` + `mach_vm_map`, `vm_purgable_control` for Discardable, IOSurface for DeviceShareable.

---

7. Threads and synchronisation

7.1 Model

A `Thread` is a kernel thread inside a `Process`. LibTessera owns thread creation end to end because there is no libc: it allocates the stack (a `Memory` object with guard pages), the thread-local storage block, sets the architectural TLS register, and runs the entry. Threads have a QoS class; realtime scheduling is an entitlement.

There is exactly one blocking synchronisation primitive: wait/wake on an address. Mutexes, condition variables, semaphores, once-cells, and parking are built on it by the runtime above (Rust std's `futex`-based `sys::sync`, Kira's runtime).

7.2 Types

```
enum Qos: UInt32
  Background = 0, Utility = 1, Default = 2, UserInitiated = 3, UserInteractive
= 4, Realtime = 5

struct ThreadOptions (40 bytes)
  stackSize:  u64 @0     0 = default (from
System.info(.Limits).defaultStackSize)
  guardSize:  u64 @8     0 = one page
  qos:        u32 @16
  flags:      u32 @20    bit0 DETACHED (no handle returned; termination still
visible in Job/Process info)
  name:       Buffer @24 ≤ 64 bytes UTF-8, may be empty

struct RealtimePolicy (24 bytes)
  period:      Duration @0    e.g. audio callback period
  computation: Duration @8    expected CPU per period
  constraint:  Duration @16   deadline within the period

struct ThreadInfo (topic Thread, v1, 48 bytes)
  threadId: u64 @0, processId: u64 @8, state: u32 @16 (Created=0, Running=1,
Blocked=2, Terminated=3),
  qos: u32 @20, exitCode: i64 @24, cpuTime: Duration @32, reserved: u64 @40
```

7.3 Calls (all synchronous)

```
Thread.spawn(entry: Address, argument: UInt64, options: ThreadOptions) ->
Result<Handle<Thread>, Error>
```

```
C  tessera_error_t tessera_thread_spawn(uint64_t entry, uint64_t argument,
const tessera_thread_options_t* options, tessera_handle_t* out);
```

`entry` is a function `fn(argument: u64) -> i64` in the platform calling convention. The thread runs immediately. Returns a handle with `Wait | Inspect | Manage | SetProperty | Duplicate` (not `Transfer`: threads never cross processes). Errors: `OutOfMemory`, `QuotaExceeded` (job thread limit), `NotEntitled` (Realtime qos).

```
Thread.exit(code: Int64) -> noreturn
C  _Noreturn void tessera_thread_exit(int64_t code);
```

Terminates the calling thread; the last thread's exit terminates the process with that code. Stack and TLS are reclaimed by LibTessera.

```
Thread.current() -> Handle<Thread>
C  tessera_handle_t tessera_thread_current(void);
```

A borrowed, never-closed handle to the calling thread (rights `Inspect | SetProperty | Manage`).

```
Thread.id(thread: Handle<Thread>) -> Result<UInt64, Error>
C  tessera_error_t tessera_thread_id(tessera_handle_t thread, uint64_t* out);
```

Process-unique, never reused within the process's life.

```
Thread.setQos(thread: Handle<Thread>, qos: Qos) -> Result<Void, Error>
Thread.setRealtime(thread: Handle<Thread>, policy: RealtimePolicy) ->
Result<Void, Error>
C  tessera_error_t tessera_thread_set_qos(tessera_handle_t thread, uint32_t
qos);
C  tessera_error_t tessera_thread_set_realtime(tessera_handle_t thread, const
tessera_realtime_policy_t* policy);
```

Requires `Manage`. `setRealtime` requires entitlement `tessera.thread.realtime` (granted with `tessera.audio.*`); the system may revoke it under thermal pressure (the thread drops to `UserInteractive`, reported via `Thread.info`).

```
Thread.sleep(deadline: Deadline) -> Result<Void, Error>
Thread.yield() -> Void
C  tessera_error_t tessera_thread_sleep(uint64_t deadline);
C  void tessera_thread_yield(void);
```

```
Thread.kill(thread: Handle<Thread>) -> Result<Void, Error>
C  tessera_error_t tessera_thread_kill(tessera_handle_t thread);
```

Requires `Manage`. Terminates the thread at the next safe point without running any unwinding. Its stack is leaked until process exit. Exists for runtimes that need it (debuggers, watchdogs); ordinary code joins.

Join is `Object.waitAsync(thread, TERMINATED, …)`; the exit code is `Thread.info.exitCode`.

```
Thread.tlsBase() -> Address
Thread.setTlsBase(base: Address) -> Result<Void, Error>
C  uint64_t tessera_thread_tls_base(void);
C  tessera_error_t tessera_thread_set_tls_base(uint64_t base);
```

Read/write the architectural thread pointer (`TPIDR_EL0` on ARM64, `FS` on x64). `Thread.spawn` allocates and initialises a TLS block from the executable's TLS template (§11.5) before the entry runs, so `#[thread_local]` and Kira thread-locals work with no runtime setup; runtimes that manage their own TLS may replace it.

7.4 Wait/wake on address

```
Sync.wait32(address: Address, expected: UInt32, deadline: Deadline) ->
Result<Void, Error>
Sync.wait64(address: Address, expected: UInt64, deadline: Deadline) ->
Result<Void, Error>
C  tessera_error_t tessera_sync_wait32(uint64_t address, uint32_t expected,
uint64_t deadline);
C  tessera_error_t tessera_sync_wait64(uint64_t address, uint64_t expected,
uint64_t deadline);
```

Atomically checks `*address == expected` and, if so, blocks until woken or the deadline passes. Returns `Ok` when woken (spurious wakeups are permitted, callers loop), `BadState` if the value differed, `Timeout` on deadline. The address must be naturally aligned and mapped `Read`.

```
Sync.wake(address: Address, count: UInt32) -> Result<UInt32, Error>
C  tessera_error_t tessera_sync_wake(uint64_t address, uint32_t count,
uint32_t* out_woken);
```

Wakes up to `count` waiters (`UInt32.max` = all). Works across processes when the address is in shared `Memory` mapped by both.

```
Sync.requeue(address: Address, wakeCount: UInt32, target: Address,
requeueCount: UInt32) -> Result<UInt32, Error>
C  tessera_error_t tessera_sync_requeue(uint64_t address, uint32_t wake_count,
uint64_t target, uint32_t requeue_count, uint32_t* out_woken);
```

Wakes `wakeCount` waiters and moves up to `requeueCount` others to wait on `target` (condition-variable broadcast without the thundering herd).

7.5 Backend

Linux: `clone3(CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_CHILD_CLEARTID)` with the stack from a `Memory` mapping; QoS = `sched_setattr` (SCHED_OTHER nice / `SCHED_DEADLINE` for Realtime); wait/wake = `futex2` / `futex` (`FUTEX_WAIT_BITSET` with absolute `CLOCK_MONOTONIC` deadline, `FUTEX_REQUEUE`); `kill` = `tgkill` with an internal reserved signal handled only inside LibTessera's stub (the only signal Tessera ever installs). XNU: `bsdthread_create` / `thread_create` + `thread_set_state`, `__ulock_wait2` / `__ulock_wake`, `thread_policy_set` for QoS and `THREAD_TIME_CONSTRAINT_POLICY` for realtime.

---

8. Time

8.1 Clocks

```
enum Clock: UInt32
  Monotonic  = 0   never jumps, stops during suspend; all Deadlines are on this
clock
  Boottime   = 1   like Monotonic but advances during suspend
  Realtime   = 2   nanoseconds since 1970-01-01T00:00:00Z (UTC); read-only for
apps; may jump
  ProcessCpu = 3   CPU consumed by the calling process
  ThreadCpu  = 4   CPU consumed by the calling thread
```

Time zones, locale, calendar and formatting are not LibTessera concerns: apps read UTC here and get the zone from the Settings service; apps can never set any clock (Settings redirect, §14.6).

```
Time.read(clock: Clock) -> Result<Instant, Error>
Time.resolution(clock: Clock) -> Result<Duration, Error>
C  tessera_error_t tessera_time_read(uint32_t clock, uint64_t* out);
C  tessera_error_t tessera_time_resolution(uint32_t clock, uint64_t* out);
```

`Time.read` is a vDSO-class fast path (no trap on Linux/XNU).

```
Time.realtimeDetails() -> Result<RealtimeDetails, Error>
C  tessera_error_t tessera_time_realtime_details(tessera_realtime_details_t*
out);
struct RealtimeDetails (32 bytes)
  monotonicOffset: i64 @0   Realtime − Monotonic at the moment of the call
  synchronised: u8 @8       1 if the clock was set from a trusted source
  reserved: [u8; 7] @9
  lastSet: Instant @16      Monotonic instant of the last adjustment
  errorBound: Duration @24  estimated error, 0 if unknown
```

8.2 Timers

```
Timer.create(clock: Clock) -> Result<Handle<Timer>, Error>
C  tessera_error_t tessera_timer_create(uint32_t clock, tessera_handle_t* out);
```

`clock` must be `Monotonic` or `Boottime`. A `Boottime` timer with the `Wake` option can wake the device from suspend (entitlement `tessera.power.wake`).

```
Timer.set(timer: Handle<Timer>, deadline: Instant, slack: Duration, options:
TimerOptions, port: Handle<Port>, key: Key) -> Result<Void, Error>
C  tessera_error_t tessera_timer_set(tessera_handle_t timer, uint64_t deadline,
uint64_t slack, uint32_t options, tessera_handle_t port, uint64_t key);
enum TimerOptions: UInt32 (flags)   None = 0, Wake = 1 << 0
```

One-shot. Arms (or re-arms, replacing any pending arming) the timer. Completion kind `Timer`, `result0` = 1, `result1` = actual fire instant. `slack` lets the system coalesce timers; `0` means precise. Cancelling delivers `Cancelled`.

```
Timer.setPeriodic(timer: Handle<Timer>, first: Instant, period: Duration,
slack: Duration, options: TimerOptions, port: Handle<Port>, key: Key) ->
Result<Void, Error>
C  tessera_error_t tessera_timer_set_periodic(tessera_handle_t timer, uint64_t
```

```
first, uint64_t period, uint64_t slack, uint32_t options, tessera_handle_t
port, uint64_t key);
```

Multi-shot: one completion per period, `result0` = number of expirations since the previous packet (>1 if the waiter fell behind), `flags.MORE` set.

```
Timer.cancel(timer: Handle<Timer>) -> Result<Bool, Error>
C  tessera_error_t tessera_timer_cancel(tessera_handle_t timer, uint8_t*
out_was_armed);
```

Backend: Linux `TIMEOUT` / `TIMEOUT_REMOVE` SQEs for precise, `timerfd` (`TFD_TIMER_ABSTIME`, `CLOCK_BOOTTIME_ALARM` for Wake) for periodic/wake; XNU `EVFILT_TIMER` with `NOTE_ABSOLUTE|NOTE_MACHTIME`.

---

9. Events

```
Event.create(initial: Signals) -> Result<Handle<Event>, Error>
C  tessera_error_t tessera_event_create(uint32_t initial, tessera_handle_t*
out);
```

A standalone signal word. `Object.signal` sets/clears `SIGNALED` and user bits; `Object.waitAsync` observes them. Used for in-process wakeups that must land on a Port (integrating a futex-based runtime with completion loops) and as a transferable "done" flag.

```
EventPair.create() -> Result<(Handle<EventPair>, Handle<EventPair>), Error>
C  tessera_error_t tessera_eventpair_create(tessera_handle_t* out_a,
tessera_handle_t* out_b);
```

Two linked ends. Signals set on one end are observable on the other; when the last handle to one end closes, the other end asserts `PEER_CLOSED`. Send one end to another process for a cheap liveness/lease token (a grant lease, a surface lease, a lock).

Backend: Linux — in-process `Event` is a futex word plus an eventfd registered with each waiting Port; `EventPair` is an `AF_UNIX` socketpair carrying a 4-byte signal word, with `POLLHUP` as `PEER_CLOSED`. XNU — Mach port pairs with `EVFILT_MACHPORT`.

10. Channels

10.1 Model

A `Channel` is one end of a bidirectional, ordered, reliable, message-oriented transport that can carry handles. It is the IPC primitive: every system service, every child-process relay, every compositor conversation is a channel. Messages are delivered whole (no partial reads) and in order per direction. Loss is impossible; when a peer dies the survivor sees `PEER_CLOSED` after draining what was already queued.

Limits (from `System.info(.Limits)`): message ≤ 64 KiB bytes and ≤ 64 handles; per-direction queue ≤ 4 MiB or 1 024 messages, after which `write` returns `ShouldWait` and `WRITABLE` deasserts — backpressure, never unbounded buffering. Larger payloads travel as a `Memory` handle inside the message.

Handle transfer semantics: handles listed in `write` are **moved** — removed from the sender's table whether the write succeeds or fails (on failure they are closed), so a caller can never hold a handle whose ownership is ambiguous. A handle without the `Transfer` right makes the whole write fail with `AccessDenied` before anything moves.

10.2 Calls (all synchronous, non-blocking; readiness via `Object.waitAsync`)

```
Channel.create() -> Result<(Handle<Channel>, Handle<Channel>), Error>
C  tessera_error_t tessera_channel_create(tessera_handle_t* out_a,
tessera_handle_t* out_b);
```

```
Channel.write(channel: Handle<Channel>, bytes: Buffer, handles: [Handle<Any>],
handleCount: UInt32) -> Result<Void, Error>
C  tessera_error_t tessera_channel_write(tessera_handle_t channel,
tessera_buffer_t bytes, const tessera_handle_t* handles, uint32_t
handle_count);
```

Requires `Write`. Errors: `ShouldWait` (queue full), `PeerClosed`, `OutOfRange` (limits), `AccessDenied` (a handle lacks Transfer), `InvalidHandle`.

```
Channel.read(channel: Handle<Channel>, bytes: Buffer, handles: [Handle<Any>],
handleCapacity: UInt32) -> Result<(Size, UInt32), Error>
C  tessera_error_t tessera_channel_read(tessera_handle_t channel,
tessera_buffer_t bytes, tessera_handle_t* handles, uint32_t handle_capacity,
uint64_t* out_bytes, uint32_t* out_handles);
```

Requires `Read`. Dequeues one whole message. `ShouldWait` if empty; `PeerClosed` if empty and the peer is gone; `BufferTooSmall` (message left queued; `detail` = bytes needed, and `Channel.peek` gives both sizes). Received handles are added to the caller's table with the rights they were sent with.

```
Channel.peek(channel: Handle<Channel>) -> Result<(Size, UInt32), Error>
C  tessera_error_t tessera_channel_peek(tessera_handle_t channel, uint64_t*
out_bytes, uint32_t* out_handles);
```

Sizes of the next message without dequeuing. `ShouldWait` if none.

```
Channel.call(channel: Handle<Channel>, request: Buffer, requestHandles:
[Handle<Any>], requestHandleCount: UInt32, response: Buffer, responseHandles:
[Handle<Any>], responseHandleCapacity: UInt32, deadline: Deadline, port:
Handle<Port>, key: Key) -> Result<Void, Error>
C  tessera_error_t tessera_channel_call(tessera_handle_t channel,
tessera_buffer_t request, const tessera_handle_t* req_handles, uint32_t
req_handle_count, tessera_buffer_t response, tessera_handle_t* resp_handles,
uint32_t resp_handle_capacity, uint64_t deadline, tessera_handle_t port,
uint64_t key);
```

Request/response in one operation. LibTessera writes a fresh transaction id into the first 8 bytes of `request` (the wire header's `txid`, §10.3), sends it, and matches the reply whose `txid` equals it — replies for other transactions are left for `Channel.read`. Asynchronous: completion kind `Service`, `result0` = response bytes, `result1` = response handles, `error` = `Timeout`, `PeerClosed`, `BufferTooSmall`, or the service's own error copied from an error reply. Many calls may be outstanding on one channel from many threads.

10.3 Tessera IPC wire format v1

Every message on a service channel begins with a 24-byte header; the body follows, encoded by the rules below. Protocols are declared in Kira as `protocol` constructs and the encoder/decoder is generated; the wire format is the stable contract, the Kira declaration is the source of truth.

```
struct MessageHeader (24 bytes)
  txid:    u64 @0    transaction id (0 for unsolicited events); set by
Channel.call or the client
  ordinal: u64 @8    FNV-1a 64 of "ServiceName/MethodName"; a new signature is
a new name
  flags:   u32 @16   bit0 RESPONSE, bit1 ERROR (body is an Error), bit2 EVENT
(server→client, no reply)
  version: u32 @20   protocol CalVer as YYYYMM (e.g. 202610); receivers accept
any version whose methods they know
```

Encoding: little-endian; fields in declaration order; each field aligned to its natural alignment within 8-byte-aligned blocks; `Bool` is `u8`; enums `u32`; `String` / `Bytes` / `Vector<T>` = `u64 count` followed by inline elements padded to 8; `Optional<T>` = `u64 present` (0/1) followed by `T` if present; nested structs inline; `Handle<T>` = `u32 index` into the message's handle array (`0xFFFF_FFFF` = none), and handles appear in the array in field order; `Number` is forbidden. An error reply's body is exactly one `Error` (16 bytes).

Example declaration (illustrative Kira):

```
@Protocol("tessera.resolver", introduced: "2026.10")
protocol Resolver {
    func resolve(name: String, family: AddressFamily) ->
Result<Vector<NetAddress>, Error>
    event addressesChanged(interface: String)
}
```

10.4 Backend

Linux: `socketpair(AF_UNIX, SOCK_SEQPACKET | SOCK_CLOEXEC)`, handles via `SCM_RIGHTS` (each transferred handle is re-materialised from its fd plus a 24-byte descriptor record LibTessera prepends to the datagram: object type, rights, object id — the receiver's LibTessera reconstructs the typed handle; rights on the descriptor are trusted only from the system side, i.e. peer credentials checked via `SO_PEERCRED`), `SO_SNDBUF` / `SO_RCVBUF` sized to the queue limit, `POLLHUP` as `PEER_CLOSED`. XNU: Mach ports with `MACH_MSG_TYPE_MOVE_SEND` for handle transfer and port-death notifications for `PEER_CLOSED`. Native kernel: channels are first-class objects.

---

11. Processes and jobs

11.1 Model

- A **Job** is a node in a tree. It owns processes and child jobs, carries a policy (limits), and dies as a unit: killing a job kills every process and job beneath it. The launcher creates one job per app session; the app may create child jobs for helpers.
- A **Process** is an address space, a handle table and a set of threads. It belongs to exactly one job. It has an identity assigned by the Launcher service: `appId` (bundle), `processId`, and whether it is the app's **main process** — the only one that receives a `Presence` handle.
- There is no `fork`. A process is created by `Process.launch` from an executable `Node` inside the app bundle (or a system tool the caller is entitled to), with an explicit list of handles it will receive and nothing else.
- `Process.launch` is brokered by the Launcher service so that identity, entitlement-derived sandbox and the child's `System` channel are minted by the system, never by the parent. The parent gets a private channel to the child and a `Process` handle.

11.2 Types

```
struct LaunchOptions (64 bytes)
  name:            Buffer   @0    diagnostic name (≤ 64 bytes)
  arguments:       Handle<Memory>? @16   a Memory whose contents are the
LaunchInfo block (§11.4); 0 = none
  initialHandles:  Address  @24   pointer to [StartupEntry]
  initialCount:    u32      @32
  flags:           u32      @36   bit0 SUSPENDED (create stopped; start with
Process.resume), bit1 INHERIT_QOS
  job:             Handle<Job>? @40   job to place the child in; 0 = caller's
job
  stackSize:       u64      @48   main-thread stack, 0 = default
  reserved:        u64      @56

struct StartupEntry (16 bytes)
  tag:     u32 @0      StartupTag
  rights:  u32 @4      0 = pass as-is; otherwise a Rights mask (low 32 bits) to
narrow to
  handle:  Handle @8   moved into the child (consumed on submission, like
Channel.write)

enum StartupTag: UInt32
```

```
  System        = 1    Channel to tessera.system — minted by the Launcher,
cannot be supplied by the parent
  Presence      = 2    Presence — main process only, minted by the Launcher
  ContainerData = 3    Node: app-private data (Read|Write|Enumerate|Create|
Delete|Traverse)
  ContainerCache= 4    Node: app-private cache (may be purged by the system)
  ContainerTemp = 5    Node: app-private temp (purged at session end)
  Bundle        = 6    Node: the app bundle, read-only (Read|Enumerate|
Traverse)
  Entitlements  = 7    Memory: read-only page(s) holding the process's
EntitlementSet (§11.6)
  Launch        = 8    Memory: LaunchInfo block (arguments, launch reason, deep
link)
  StdIn         = 9    Channel or File
  StdOut        = 10   Channel or File
  StdErr        = 11   Channel or File
  ParentChannel = 12   Channel: child end of the parent↔child private channel
  Job           = 13   Job: the process's own job (Inspect only unless the
parent passes more)
  Executable    = 14   Node: the executable that was launched (Read)
  // 15–4095 reserved for Tessera; 4096+ free for apps (their own helper
protocols)
```

```
struct ProcessInfo (topic Process, v1, 96 bytes)
  processId: u64 @0, jobId: u64 @8, appId: u64 @16 (stable hash of bundle id),
  state: u32 @24 (Created=0, Running=1, Suspended=2, Terminated=3), isMain: u8
@28, reserved0: [u8;3] @29,
  exitCode: i64 @32, startTime: Instant @40 (Boottime), cpuTime: Duration @48,
  threadCount: u32 @56, handleCount: u32 @60, residentBytes: u64 @64,
mappedBytes: u64 @72,
  terminationReason: u32 @80 (Exited=0, Killed=1, Crashed=2, JobKilled=3,
OutOfMemory=4), reserved1: u32 @84, reserved2: u64 @88

struct JobPolicy (64 bytes)
  maxProcesses: u32 @0, maxThreads: u32 @4, maxHandlesPerProcess: u32 @8,
reserved0: u32 @12,
  memoryLimit: u64 @16 (bytes, 0 = inherit), cpuShare: u32 @24 (relative
weight, 0 = inherit), maxQos: u32 @28,
  flags: u32 @32 (bit0 DENY_LAUNCH: no further Process.launch beneath; bit1
DENY_NETWORK; bit2 DENY_JIT),
  reserved1: [u32;7] @36

struct JobInfo (topic Job, v1, 48 bytes)
  jobId: u64 @0, parentJobId: u64 @8, processCount: u32 @16, childJobCount: u32
@20,
  state: u32 @24 (Active=0, Killing=1, Dead=2), reserved: u32 @28,
residentBytes: u64 @32, cpuTime: Duration @40
```

11.3 Calls

```
Process.launch(executable: Handle<Node>, options: LaunchOptions, port:
Handle<Port>, key: Key) -> Result<Void, Error>
C  tessera_error_t tessera_process_launch(tessera_handle_t executable, const
tessera_launch_options_t* options, tessera_handle_t port, uint64_t key);
```

Asynchronous (Launcher service). Requires `Read | Execute` on `executable`, entitlement `tessera.process.launch` (default for apps that declare helpers; terminals/IDEs/build tools declare it), and the caller's job must not have `DENY_LAUNCH`. Completion kind `Launch`: `result0` = `Handle<Process>` (rights `Wait | Inspect | Manage | Duplicate`), `result1` = `Handle<Channel>` (parent end of the private channel), `aux` = child `processId`. Errors: `NotEntitled`, `AccessDenied`, `QuotaExceeded`, `InvalidArgument` (a `StartupEntry` uses a reserved tag the parent may not supply), `NotFound` (executable not in this bundle and not an entitled system tool), `OutOfMemory`.

The Launcher: verifies the executable is inside the caller's bundle (or an entitled system tool), derives the child's entitlement set (the parent's set ∩ the executable's declared set — a child can never exceed its parent), builds the child's sandbox from that set, mints the child's `System` channel with identity `{appId, processId, isMain: false}`, creates the process in the requested job, assembles the `Startup` message from the Launcher-owned entries plus the parent's, writes it to the child's bootstrap channel, and starts the main thread.

```
Process.exit(code: Int64) -> noreturn
C  _Noreturn void tessera_process_exit(int64_t code);
```

Terminates all threads immediately (no unwinding), closes all handles, frees the address space.

```
Process.current() -> Handle<Process>
C  tessera_handle_t tessera_process_current(void);
```

Borrowed handle to the calling process (`Inspect | SetProperty`).

```
Process.kill(process: Handle<Process>) -> Result<Void, Error>
Process.suspend(process: Handle<Process>) -> Result<Void, Error>
Process.resume(process: Handle<Process>) -> Result<Void, Error>
C  tessera_error_t tessera_process_kill(tessera_handle_t process);
```

```
C  tessera_error_t tessera_process_suspend(tessera_handle_t process);
C  tessera_error_t tessera_process_resume(tessera_handle_t process);
```

Require `Manage`. Kill is immediate and unconditional; the process's `TERMINATED` signal asserts and `ProcessInfo.terminationReason = Killed`. Suspend stops every thread at a safe point (used by the system for background freezing and by debuggers in the debug image).

```
Job.create(parent: Handle<Job>, policy: JobPolicy) -> Result<Handle<Job>,
Error>
Job.kill(job: Handle<Job>) -> Result<Void, Error>
Job.setPolicy(job: Handle<Job>, policy: JobPolicy) -> Result<Void, Error>
C  tessera_error_t tessera_job_create(tessera_handle_t parent, const
tessera_job_policy_t* policy, tessera_handle_t* out);
C  tessera_error_t tessera_job_kill(tessera_handle_t job);
C  tessera_error_t tessera_job_set_policy(tessera_handle_t job, const
tessera_job_policy_t* policy);
```

`create` requires `Manage` on `parent`; a child job's policy may only tighten its parent's. `setPolicy` may only tighten (loosening returns `AccessDenied`). Job handles carry `Transfer` so a parent can hand a helper a job handle limited to `Inspect`.

Waiting for a process or job is `Object.waitAsync(handle, TERMINATED, …)`; the exit code and reason are in `ProcessInfo`.

11.4 The bootstrap protocol

A new process begins with exactly one handle: its **bootstrap channel**, delivered by the backend in a kernel-specific way (§21) and registered by LibTessera's entry stub as the first entry in the handle table. The first (and only) message on it is the `Startup` message:

```
struct StartupMessage (variable)
  magic:      u32 @0     0x54535254 ("TSRT")
  version:    u32 @4     1
  entryCount: u32 @8
  flags:      u32 @12    bit0 IS_MAIN_PROCESS, bit1 DEBUG_IMAGE, bit2
CERTIFIED_DEVICE
  appId:      u64 @16
  processId:  u64 @24
  jobId:      u64 @32
  reserved:   u64 @40
```

```
  entries:    [StartupEntryWire; entryCount] @48   {tag: u32, handleIndex: u32}
— handleIndex into the message's handle array
```

`LaunchInfo` (in the `Launch` Memory) is encoded with the §10.3 rules:

```
struct LaunchInfo
  reason:     u32                Cold=0, Resume=1, Url=2, Intent=3, Helper=4,
Debug=5
  arguments:  Vector<String>     argv-style, empty for GUI launches
  url:        Optional<String>   for Url launches
  intent:     Optional<Bytes>    opaque intent payload
  locale:     String             BCP-47 tag chosen by Settings (read-only for
the app)
  workingDirectoryHint: Optional<String>   for command-line tools: the
*display* path the user was in; not authority
```

11.5 Runtime initialisation sequence (no libc)

LibTessera ships the process entry point for each backend/architecture — the only assembly in the system:

1. Kernel enters `_tessera_start` with the initial stack. The stub aligns the stack, captures the bootstrap descriptor, and calls `tessera_runtime_init(descriptor)`.
2. `tessera_runtime_init`: builds the handle table, registers the bootstrap channel, reads the `Startup` message, materialises its handles, maps the `Entitlements` page, initialises the main thread's TLS block from the executable's TLS template (`PT_TLS` in ELF / `__thread_vars` in Mach-O — the *only* place the executable format is known, inside the backend), sets the thread pointer, initialises the per-thread Port slot, and installs the backend's internal signal stub (Linux only; see §21.2).
3. Calls the executable's `tessera_main(startup: *const StartupTable) -> i64`, where `StartupTable` is the decoded message (tags → handles). The Rust std's `lang_start` and Kira's runtime entry are `tessera_main`.
4. On return, `Process.exit(returnValue)`.

There is no dynamic loader: Tessera apps arrive as stable Kira IR and are AOT-linked by the installer against the current system libraries, so the executable is a single static image whose only external symbols are LibTessera's, resolved at install time. System frameworks (KiraUI, Kira Graphics) are linked the same way — the "shared library" is an install-time artefact, not a runtime one. (Off-platform Kira apps bundle their runtime; that is outside this document.)

11.6 Entitlements

The `Entitlements` Memory page is a read-only, §10.3-encoded `EntitlementSet`: `Vector<Entitlement { name: String, value: Optional<Bytes> }>`, sorted by name, minted by the Launcher from the bundle's notarised declaration ∩ the parent's set. Consumers check it locally (`Entitlement.has(name)` is a Rust-std/Kira-runtime helper over the page, not a syscall); the backend sandbox (§21) enforces it. `tessera.system.EntitlementChanged` announces revocation at runtime; the page is replaced atomically and the old mapping keeps reading until unmapped.

Well-known entitlements referenced in this document: `tessera.process.launch`, `tessera.jit`, `tessera.thread.realtime`, `tessera.network.client`, `tessera.network.server`, `tessera.network.local`, `tessera.network.raw`, `tessera.sensors.motion`, `tessera.sensors.environment`, `tessera.audio.capture`, `tessera.audio.playback` (default), `tessera.camera`, `tessera.haptics` (default), `tessera.gpu` (default), `tessera.driver` (system only), `tessera.power.wakelock`, `tessera.power.wake`, `tessera.container.shared.<id>`, `tessera.debug.crash` (debug image), `tessera.trace` (default).

11.7 Backend

Linux: process = `clone3(CLONE_VFORK|CLONE_VM…)` in the Launcher followed by `execveat` of the AOT image, or `CLONE_INTO_CGROUP` into the job's cgroup; `pidfd` for the Process handle (`TERMINATED` = pidfd readable); Job = cgroup v2 subtree (limits map to `memory.max`, `pids.max`, `cpu.weight`) plus a kill via `cgroup.kill`; bootstrap channel = fd 0 (the only inherited fd, `O_CLOEXEC` on everything else); sandbox = user namespace + mount namespace containing only the container directories, seccomp filter generated from the entitlement set, Landlock as belt-and-braces on the container roots. XNU: `posix_spawn` from the Launcher with a special port for bootstrap, `proc_info` for info, process death via `EVFILT_PROC`, Jobs via `coalition`s and `memorystatus`, sandbox via `sandbox_init` profiles generated from the entitlement set.

---

12. The System channel protocol (`tessera.system`)

Every process holds a `System` channel (tag 1). It is the process's line to the OS and the root of service discovery. Protocol methods (client → system, via `Channel.call`) and events (system → client, `flags.EVENT`):

```
@Protocol("tessera.system", introduced: "2026.10")
protocol System {
    // discovery
    func connectService(name: String) -> Result<Handle<Channel>, Error>
        // NotEntitled if the service requires an entitlement the process
lacks; NotFound if unknown
    func listServices() -> Result<Vector<ServiceInfo { name: String,
requiredEntitlement: Optional<String> }>, Error>

    // lifecycle (events carry a sequence number; the app acks the ones marked
needsAck)
    event lifecycle(seq: UInt64, state: LifecycleState, deadline:
Optional<Instant>, needsAck: Bool)
        // LifecycleState: Foreground, Background, Suspending, Resumed,
Terminating
    func ackLifecycle(seq: UInt64) -> Result<Void, Error>
    event memoryPressure(level: MemoryPressureLevel)          // Normal,
Warning, Critical
    event terminateSoon(deadline: Instant, reason: UInt32)     // the app has
until deadline to flush

    // grants (see §14)
    event grantDelivered(requestId: UInt64, node: Handle<Node>, grantId:
GrantId)   // for grants brokered for this process by its main process
    event grantRevoked(grantId: GrantId, reason: UInt32)
    event grantExpired(grantId: GrantId)

    // policy changes
    event entitlementChanged(entitlements: Handle<Memory>)     // replacement
page
    event powerChanged(info: PowerInfo)
    event thermalChanged(level: ThermalLevel)
    event networkChanged(summary: NetworkSummary)
    event deviceChanged(class: DeviceClass, deviceId: UInt64, added: Bool)
    event localeChanged(locale: String)
    event timeChanged(details: RealtimeDetails)                // Realtime
clock jumped
}
```

Well-known services (reached with `connectService`):

Name	Purpose	Entitlement
`tessera.namespace`	filesystem namespace operations (§13)	none (containers), grants gate the rest
`tessera.presence`	grants, pickers, authorization prompts, settings redirect (§14)	holding `Presence`

Name	Purpose	Entitlement
`tessera.launcher`	process launch (§11)	`tessera.process.launch`
`tessera.devices`	device enumeration and streams (§16)	per class
`tessera.resolver`	DNS / name resolution	`tessera.network.client`
`tessera.network`	interfaces, connectivity, proxy settings	none (read-only)
`tessera.display`	MosaicDisplay: surfaces, buffers, presentation, input routing to KCoreSurface	none (GUI apps)
`tessera.passwords`	Passwords API (agents fill by token; secrets never cross)	`tessera.passwords`
`tessera.power`	wake locks, scheduled wake (§18)	`tessera.power.*`
`tessera.audit`	read your own grant/request history (§14.7)	none
`tessera.settings`	read settings; never write (redirect)	none
`tessera.agent`	agent Control API (separate desktop), Accessibility read side	`tessera.agent.*`
`tessera.debug`	crash reports, on-device diagnostics (debug image only)	`tessera.debug.*`

Service channels are ordinary channels: `PEER_CLOSED` means the service restarted and the client reconnects via `connectService`. Services are resolved by name at connect time, which is what makes the set "changeable enough": a service can be replaced, split, or re-implemented without touching LibTessera, and its protocol evolves by adding methods (new ordinals) under CalVer.

---

13. Namespace and files

13.1 Model

The user owns the filesystem. A process holds exact capabilities:

- A **Node** is a namespace capability naming one file or directory, with rights `Read | Enumerate | Create | Write | Delete | Traverse` (the five permission-model operations plus Traverse). A Node on a directory covers *that directory and, with Traverse, its subtree*, scoped exactly as the grant that produced it.
- A **File** is a data handle to a file's bytes, obtained from a Node with `File.open`; it carries `Read`/`Write`/`Map` and nothing about names. Data I/O never touches the Namespace service.
- A **Watch** is a change-notification stream on a Node.
- Every namespace operation — lookup, list, create, delete, rename, attributes — goes through the **Namespace service** over `tessera.namespace`; LibTessera wraps this so callers see ordinary asynchronous calls. Every data operation — read, write, sync, resize, map — goes straight to the kernel on a File handle.
- Processes never see absolute paths. All paths are relative to a Node. Display paths for UI come from `Node.info(.Display)` and carry no authority.
- App-private containers (`ContainerData/Cache/Temp`) arrive at startup with full rights. Everything else requires a grant (§14): temporary, session, persistent, revocable, expirable, audited.
- Shared containers between apps are explicit (`tessera.container.shared.<id>` entitlement, `SharedContainer.open`). Two apps never see each other's data because of a common kernel user.
- The system side is immutable and simply absent from any app's namespace. Protected roots (a user's home root, container roots) refuse `Delete` with `ProtectedNode`; removing an account is a privileged system action outside this API.

13.2 Path rules

A `Path` is a `Buffer` of UTF-8, NFC-normalised, with these rules (violations return `InvalidPath`, `detail` = byte offset):

- components separated by `/`; no leading, trailing or doubled `/`; empty path means "this node";

- no component may be `.` or `..`; escaping a node's scope is impossible by construction;
- component ≤ 255 bytes, path ≤ 4 096 bytes; no NUL, no control characters U+0000–U+001F;
- case-sensitive, case-preserving; the Namespace service may additionally reject names that collide case-insensitively inside user-visible locations (policy, reported as `AlreadyExists`);
- symbolic links are not exposed: a symlink in a granted subtree is reported as `NodeKind.Link` and cannot be traversed (`NotDirectory`). Hard links are not creatable.

13.3 Types

```
enum NodeKind: UInt32   File = 0, Directory = 1, Link = 2, Selection = 3
(virtual directory of picker results), Special = 4

struct NodeInfo (topic Node, v1, 128 bytes)
  kind:        u32 @0
  flags:       u32 @4      bit0 PROTECTED_ROOT, bit1 CONTAINER_ROOT, bit2
GRANTED (not a container), bit3 SHARED, bit4 READ_ONLY_VOLUME
  size:        u64 @8      bytes (files); entry count hint (directories)
  nodeId:      u64 @16     stable identity of the underlying node on its volume
  volumeId:    u64 @24
  created:     Instant @32 (Realtime)
  modified:    Instant @40
  accessed:    Instant @48
  changed:     Instant @56  metadata change
  rights:      Rights @64   effective rights of this handle
  grantId:     u128 @72     0 for containers
  grantExpiry: Instant @88  Boottime; 0 = none
  lifetime:    u32 @96      GrantLifetime (§14)
  attributes:  u32 @100     bit0 HIDDEN, bit1 IMMUTABLE, bit2 EXECUTABLE
(bundle code), bit3 ENCRYPTED_AT_REST
  blockSize:   u32 @104
  reserved0:   u32 @108
  allocated:   u64 @112     bytes actually allocated (sparse files)
  reserved1:   u64 @120

struct NodeAttributes (32 bytes)   for Node.setAttributes; each field applied
if its bit is set in `mask`
  mask:      u32 @0    bit0 MODIFIED, bit1 ACCESSED, bit2 HIDDEN, bit3
IMMUTABLE
  attributes:u32 @4    HIDDEN/IMMUTABLE values
  modified:  Instant @8
  accessed:  Instant @16
  reserved:  u64 @24
```

```
struct DirEntry (variable, 8-byte aligned)
  entrySize: u16 @0    total bytes including name and padding
  kind:      u8  @2    NodeKind
  flags:     u8  @3    bit0 HIDDEN
  nameLength:u16 @4
  reserved:  u16 @6
  nodeId:    u64 @8
  size:      u64 @16
  modified:  Instant @24
  name:      [u8; nameLength] @32, padded to 8

enum CreateKind: UInt32   File = 0, Directory = 1
enum CreateOptions: UInt32 (flags)   None = 0, Exclusive = 1 << 0 (fail if
exists), OpenExisting = 1 << 1 (return existing file's Node if present;
requires Read on it)
enum DeleteOptions: UInt32 (flags)   None = 0, Recursive = 1 << 0, Permanent =
1 << 1 (bypass the volume's trash policy if the user allows it; otherwise
ignored)
enum RenameOptions: UInt32 (flags)   None = 0, Replace = 1 << 0 (atomically
replace an existing destination; requires Delete on it), Exchange = 1 << 1
(atomic swap)

enum OpenMode: UInt32 (flags)   Read = 1 << 0, Write = 1 << 1, Append = 1 << 2,
Truncate = 1 << 3, NoCache = 1 << 4 (direct I/O), Sync = 1 << 5 (every write
durable)
enum SyncOptions: UInt32   Data = 0 (data + size), All = 1 (data + all
metadata)
enum LockMode: UInt32       Shared = 0, Exclusive = 1
enum WatchMask: UInt32 (flags)   Created = 1<<0, Deleted = 1<<1, Modified =
1<<2, Renamed = 1<<3, AttributesChanged = 1<<4, Recursive = 1<<8 (whole
subtree; requires Traverse)

struct WatchEvent (variable, 8-byte aligned)
  eventSize: u16 @0, mask: u16 @2 (which WatchMask bits), nameLength: u16 @4,
cookie: u16 @6 (pairs the two halves of a rename), nodeId: u64 @8, name: [u8]
@16 padded
```

13.4 Namespace calls (asynchronous, kind `Namespace`)

```
Node.lookup(base: Handle<Node>, path: Path, rights: Rights, port: Handle<Port>,
key: Key) -> Result<Void, Error>
C  tessera_error_t tessera_node_lookup(tessera_handle_t base, tessera_buffer_t
path, uint64_t rights, tessera_handle_t port, uint64_t key);
```

Resolves `path` beneath `base` and returns a Node with `rights` (must be ⊆ base's rights; `RIGHTS_SAME` = inherit). Requires `Traverse` on `base` to descend into directories; the final component needs no right to be *named*, but the returned

Node's rights are capped by the base. `result0` = `Handle<Node>`. Errors: `NotFound`, `InvalidPath`, `NotDirectory`, `AccessDenied`, `GrantExpired/Revoked`.

```
Node.list(directory: Handle<Node>, cursor: UInt64, out: Buffer, port:
Handle<Port>, key: Key) -> Result<Void, Error>
C  tessera_error_t tessera_node_list(tessera_handle_t directory, uint64_t
cursor, tessera_buffer_t out, tessera_handle_t port, uint64_t key);
```

Requires `Enumerate`. Fills `out` with packed `DirEntry` records. `result0` = bytes written, `result1` = next cursor (`0` = end). Listing is a snapshot per call, not per cursor sequence; concurrent changes may cause omissions or repeats, as on every OS. Names only — a Node for an entry is obtained with `lookup`, which is where rights are decided.

```
Node.create(directory: Handle<Node>, name: Path, kind: CreateKind, options:
CreateOptions, rights: Rights, port: Handle<Port>, key: Key) -> Result<Void,
Error>
C  tessera_error_t tessera_node_create(tessera_handle_t directory,
tessera_buffer_t name, uint32_t kind, uint32_t options, uint64_t rights,
tessera_handle_t port, uint64_t key);
```

Requires `Create`. `name` is a single component. Returns a Node to the new item with `rights` ⊆ directory's rights (`Create` alone lets you make the file; writing into it additionally needs `Write` on the directory — this is the "Create does not imply Update" rule made mechanical). `result0` = `Handle<Node>`. Errors: `AlreadyExists`, `NoSpace`, `QuotaExceeded`, `AccessDenied`.

```
Node.delete(directory: Handle<Node>, name: Path, options: DeleteOptions, port:
Handle<Port>, key: Key) -> Result<Void, Error>
C  tessera_error_t tessera_node_delete(tessera_handle_t directory,
tessera_buffer_t name, uint32_t options, tessera_handle_t port, uint64_t key);
```

Requires `Delete` on `directory`; `Recursive` additionally requires `Delete | Traverse` covering the subtree. Deleting a protected root (or naming one via a grant) fails `ProtectedNode`. Open File handles on a deleted file keep working until closed (the data lives until the last reference).

```
Node.rename(fromDirectory: Handle<Node>, fromName: Path, toDirectory:
Handle<Node>, toName: Path, options: RenameOptions, port: Handle<Port>, key:
Key) -> Result<Void, Error>
C  tessera_error_t tessera_node_rename(tessera_handle_t from_dir,
```

```
tessera_buffer_t from_name, tessera_handle_t to_dir, tessera_buffer_t to_name,
uint32_t options, tessera_handle_t port, uint64_t key);
```

Requires `Delete` on `fromDirectory` and `Create` on `toDirectory` (both may be the same Node). Atomic within a volume; across volumes fails `Unsupported` (copy-then-delete is the caller's decision, never silent).

```
Node.info(node: Handle<Node>, out: Buffer, port: Handle<Port>, key: Key) ->
Result<Void, Error>
C  tessera_error_t tessera_node_info(tessera_handle_t node, tessera_buffer_t
out, tessera_handle_t port, uint64_t key);
```

Requires `Inspect`. Fetches a fresh `NodeInfo` into `out`; `result0` = bytes written. The synchronous `Object.info(node, .Node)` returns the last snapshot LibTessera cached without a round trip; use `Node.info` when freshness matters.

```
Node.setAttributes(node: Handle<Node>, attributes: NodeAttributes, port:
Handle<Port>, key: Key) -> Result<Void, Error>
C  tessera_error_t tessera_node_set_attributes(tessera_handle_t node, const
tessera_node_attributes_t* attributes, tessera_handle_t port, uint64_t key);
```

Requires `SetProperty` and `Write`.

```
Node.displayPath(node: Handle<Node>, out: Buffer, port: Handle<Port>, key: Key)
-> Result<Void, Error>
C  tessera_error_t tessera_node_display_path(tessera_handle_t node,
tessera_buffer_t out, tessera_handle_t port, uint64_t key);
```

Requires `Inspect`. A user-facing, localised path string ("Documents/Reports/Q3.pdf") for UI only; never accepted as input anywhere.

```
Node.watch(node: Handle<Node>, mask: WatchMask) -> Result<Handle<Watch>, Error>
C  tessera_error_t tessera_node_watch(tessera_handle_t node, uint32_t mask,
tessera_handle_t* out);
Watch.read(watch: Handle<Watch>, out: Buffer) -> Result<Size, Error>
C  tessera_error_t tessera_watch_read(tessera_handle_t watch, tessera_buffer_t
out, uint64_t* out_bytes);
```

`watch` requires `Enumerate` (directories) or `Read` (files) and is synchronous (the subscription is set up on the service channel lazily). `Watch.read` is non-blocking and returns packed `WatchEvent`s; wait for `READABLE`. If events were dropped the

Watch asserts `OVERRUN` and `Watch.read` returns `Overrun` once; the consumer rescans.

```
SharedContainer.open(id: Buffer, rights: Rights, port: Handle<Port>, key: Key)
-> Result<Void, Error>
C  tessera_error_t tessera_shared_container_open(tessera_buffer_t id, uint64_t
rights, tessera_handle_t port, uint64_t key);
```

Requires entitlement `tessera.container.shared.<id>`. `result0` = `Handle<Node>` for the container root.

13.5 File calls (data path)

```
File.open(node: Handle<Node>, mode: OpenMode, port: Handle<Port>, key: Key) ->
Result<Void, Error>
C  tessera_error_t tessera_file_open(tessera_handle_t node, uint32_t mode,
tessera_handle_t port, uint64_t key);
```

Asynchronous (the Namespace service performs the open and hands back the data handle). `Read` mode requires `Read`; `Write/Append/Truncate` require `Write`. `result0` = `Handle<File>` with rights `Read`/`Write` matching the mode plus `Map | Duplicate | Transfer | Wait | Inspect`. Errors: `IsDirectory`, `AccessDenied`, `Busy` (exclusively locked), `GrantExpired/Revoked`.

All remaining File calls are kernel-direct.

```
File.read(file: Handle<File>, offset: UInt64, buffers: [Buffer], bufferCount:
UInt32, port: Handle<Port>, key: Key) -> Result<Void, Error>
File.write(file: Handle<File>, offset: UInt64, buffers: [Buffer], bufferCount:
UInt32, port: Handle<Port>, key: Key) -> Result<Void, Error>
C  tessera_error_t tessera_file_read(tessera_handle_t file, uint64_t offset,
const tessera_buffer_t* buffers, uint32_t buffer_count, tessera_handle_t port,
uint64_t key);
C  tessera_error_t tessera_file_write(tessera_handle_t file, uint64_t offset,
const tessera_buffer_t* buffers, uint32_t buffer_count, tessera_handle_t port,
uint64_t key);
```

Positional, scatter/gather, no file position state (Rust's `Seek` is a number in the std). `offset = UInt64.max` on write means "append atomically" (required with `Append` mode). Completion kind `Io`, `result0` = bytes transferred (short transfers are legal; 0 on read = EOF). Errors: `NoSpace`, `QuotaExceeded`, `OutOfRange` (beyond `Limits.maxFileSize`), `Internal`.

```
File.readMemory(file: Handle<File>, offset: UInt64, memory: Handle<Memory>,
memoryOffset: UInt64, length: Size, port: Handle<Port>, key: Key) ->
Result<Void, Error>
File.writeMemory(file: Handle<File>, offset: UInt64, memory: Handle<Memory>,
memoryOffset: UInt64, length: Size, port: Handle<Port>, key: Key) ->
Result<Void, Error>
C  tessera_error_t tessera_file_read_memory(tessera_handle_t file, uint64_t
offset, tessera_handle_t memory, uint64_t memory_offset, uint64_t length,
tessera_handle_t port, uint64_t key);
C  tessera_error_t tessera_file_write_memory(tessera_handle_t file, uint64_t
offset, tessera_handle_t memory, uint64_t memory_offset, uint64_t length,
tessera_handle_t port, uint64_t key);
```

I/O directly between a file and a `Memory` object without a user buffer (zero-copy where the backend can, e.g. registered buffers / splice). Requires `Write` / `Read` on `memory` respectively.

```
File.sync(file: Handle<File>, options: SyncOptions, port: Handle<Port>, key:
Key) -> Result<Void, Error>
C  tessera_error_t tessera_file_sync(tessera_handle_t file, uint32_t options,
tessera_handle_t port, uint64_t key);
```

```
File.resize(file: Handle<File>, size: UInt64, port: Handle<Port>, key: Key) ->
Result<Void, Error>
C  tessera_error_t tessera_file_resize(tessera_handle_t file, uint64_t size,
tessera_handle_t port, uint64_t key);
```

Requires `Write`. Growing produces zeros (sparse where supported). `Busy` if a `Map` of the removed range exists.

```
File.allocate(file: Handle<File>, offset: UInt64, length: Size, options:
AllocateOptions, port: Handle<Port>, key: Key) -> Result<Void, Error>
C  tessera_error_t tessera_file_allocate(tessera_handle_t file, uint64_t
offset, uint64_t length, uint32_t options, tessera_handle_t port, uint64_t
key);
enum AllocateOptions: UInt32 (flags)   None = 0 (preallocate), PunchHole = 1 <<
0, ZeroRange = 1 << 1
```

```
File.lock(file: Handle<File>, offset: UInt64, length: Size, mode: LockMode,
port: Handle<Port>, key: Key) -> Result<Void, Error>
File.unlock(file: Handle<File>, offset: UInt64, length: Size) -> Result<Void,
Error>
C  tessera_error_t tessera_file_lock(tessera_handle_t file, uint64_t offset,
uint64_t length, uint32_t mode, tessera_handle_t port, uint64_t key);
```

```
C  tessera_error_t tessera_file_unlock(tessera_handle_t file, uint64_t offset,
uint64_t length);
```

Advisory byte-range locks owned by the File handle (not the process); closing the handle releases them. `length = 0` = to end of file. Completion when acquired; `deadline` is expressed by cancelling.

```
File.info(file: Handle<File>) -> Result<FileInfo, Error>
C  tessera_error_t tessera_file_info(tessera_handle_t file,
tessera_file_info_t* out);
struct FileInfo (topic File, v1, 48 bytes)   size: u64 @0, allocated: u64 @8,
modified: Instant @16, blockSize: u32 @24, mode: u32 @28 (OpenMode), nodeId:
u64 @32, volumeId: u64 @40
```

Synchronous (kernel `fstat`).

`Memory.map(file, …)` maps file data; `Write` mappings are write-back; `Private` gives copy-on-write.

13.6 Implementation: the Namespace service

- Runs as a system process holding the real filesystem roots. Per client process it keeps a **view**: the set of Nodes that process holds, each recorded with its scope (the real path it stands for), rights, grant id and lifetime. Nodes are channel-backed: a Node handle in the client is a LibTessera-side record `{serviceNodeId, rights, cachedInfo}` bound to the client's `tessera.namespace` channel; namespace calls are `Channel.call`s carrying `serviceNodeId`, so a forged id is meaningless (the service only honours ids in *that* channel's view).
- `lookup` is resolved by the service beneath the scope with the backend's safe-resolution primitive (Linux `openat2(RESOLVE_BENEATH | RESOLVE_NO_SYMLINKS | RESOLVE_NO_MAGICLINKS)`), never by string concatenation, and never escapes the scope.
- `File.open` is performed by the service with the flags that match the granted rights; the resulting kernel object travels back through the channel as a handle (Linux: fd via `SCM_RIGHTS`, opened `O_RDONLY`/`O_RDWR`, so `Read`/`Write` are kernel-enforced on the data path). A File handle carries no name and cannot be turned back into a Node.
- Listing and attributes are performed by the service — clients never hold a directory fd, so `Enumerate`, `Create`, `Delete`, `Traverse` are enforced entirely inside the service, on a per-Node-per-client basis. This is the point of the split: the kernel cannot express "create but not delete" per handle; the

service can, and the kernel guarantees the client has no other way in (§21.4).

- Watches are `inotify` / `fanotify` (Linux) or `EVFILT_VNODE` /FSEvents (XNU) subscriptions inside the service, forwarded over a per-Watch channel; the Watch handle *is* that channel with the `Watch` type.
- Containers are real directories under a per-app root the service creates at install; `Temp` is purged at session end, `Cache` under storage pressure (announced via `memoryPressure`-style `storagePressure` events in a later CalVer).
- Grant lifetime enforcement: on expiry or revocation the service drops the Node from the view; subsequent calls fail `GrantExpired` / `GrantRevoked`, and already-open File handles are *not* revoked (data already in hand stays; new access does not) — mirroring how a user thinks about "you can keep what I gave you, but no more".
- Audit: every request, grant, denial, expiry and revocation is written to the Audit service with `{appId, processId, requestedScope, rights, outcome, timestamp, interaction token id}`; the audit reference returned in `Error.detail` indexes this log.

13.7 Enforcement ledger for this domain

Operation	Enforced by	Mechanism
Any namespace op outside the view	kernel	client sandbox has no filesystem syscalls at all (§21.2); only the service channel exists
Read/Write on file bytes	kernel	data handle opened with matching flags by the service
Enumerate/Create/ Delete/Traverse	Namespace service	per-Node rights in the client's view
Scope containment	Namespace service	safe resolution beneath the scope root
Protected roots	Namespace service	root registry; `ProtectedNode`
Grant lifetime	Namespace + Presence services	view eviction; audit
Container isolation	kernel + service	containers are the only mounts in the client's mount namespace and the only roots in its view

14. Grants, presence and pickers

14.1 Model

- **Presence** is an object that represents an app's right to interact with the user. Exactly one handle exists per app session, delivered to the main process at startup (tag 2). It carries `Grant | Inspect`, never `Duplicate` or `Transfer`: it cannot be copied or given to a child. Every call that can raise authority through the user takes a `Presence` handle, so only the main process can make one — a child process, helper, extension, daemon or CLI tool cannot, structurally.
- An **InteractionToken** is proof that a real, hardware-originated user event reached a front-facing surface of this app. MosaicDisplay mints tokens when it delivers hardware input to a KCoreSurface, attaches them to the input event, and remembers them. A token is bound to `{appId, surfaceId, issue instant}`, single-use, and expires — by default 30 s after issue, or immediately when the surface stops being front-facing. Software-synthesised events (including the agent Control API) carry no token.
- A **grant request** is causally connected to interaction because it must present a token. The Presence service verifies the token with MosaicDisplay (issued to this app, unspent, unexpired, surface still front-facing), then shows the trusted permission UI, then mints the Node and records everything in the Audit service.
- A main process brokers for a child by naming the child's `Process` handle in the request; the Node is delivered to the *child* on its System channel (`grantDelivered`), never to the requester, so "the main process is brokering a specific grant, not lending its authority" is literally how the bytes flow.

14.2 Types

```
struct InteractionToken (32 bytes)
  id:        u128 @0      random, unguessable
  surfaceId: u64  @16     MosaicDisplay surface that received the event
  issued:    Instant @24  Boottime

typedef GrantId = u128

enum GrantLifetime: UInt32
  Once       = 0    valid for one namespace operation chain; auto-released when
```

```
the Node is closed
  Session    = 1    until the app session ends
  Persistent = 2    survives relaunch; recoverable with Grant.restore(grantId)

enum PickerMode: UInt32
  OpenFile = 0, OpenFiles = 1, OpenDirectory = 2, OpenDirectories = 3, SaveFile
= 4, SaveInto = 5 (choose a directory and a new name)

struct PickerOptions (96 bytes)
  rights:          Rights  @0    rights requested on the result (capped by what
the user chooses to allow)
  lifetime:        u32     @8
  flags:           u32     @12   bit0 ALLOW_SUBTREE (request Traverse on
directories), bit1 SHOW_HIDDEN
  forProcess:      Handle<Process>? @16
  startLocation:   Handle<Node>? @24     a Node the app already holds, as the
picker's starting point
  suggestedName:   Buffer  @32           for Save modes
  allowedKinds:    Buffer  @48           UTF-8, comma-separated UTI/MIME list;
empty = any
  title:           Buffer  @64           short prompt text shown by the trusted
UI (sanitised)
  reserved:        [u64;2] @80

struct GrantRecord (variable, §10.3 encoding)   from Grant.list / Grant.info
  grantId: GrantId, requestId: UInt64, scopeDisplay: String, kind: NodeKind,
rights: Rights,
  lifetime: GrantLifetime, issued: Instant, expires: Optional<Instant>, state:
GrantState,
  targetProcessId: UInt64, auditRef: UInt64
enum GrantState: UInt32   Active = 0, Expired = 1, Revoked = 2, Released = 3,
Denied = 4
```

14.3 Calls (asynchronous, kind `Grant`, via `tessera.presence`)

```
Grant.request(presence: Handle<Presence>, target: Handle<Node>, rights: Rights,
lifetime: GrantLifetime, forProcess: Handle<Process>?, token: InteractionToken,
port: Handle<Port>, key: Key) -> Result<Void, Error>
C  tessera_error_t tessera_grant_request(tessera_handle_t presence,
tessera_handle_t target, uint64_t rights, uint32_t lifetime, tessera_handle_t
for_process, const tessera_interaction_token_t* token, tessera_handle_t port,
uint64_t key);
```

Asks the user to widen rights on a Node the app already holds (e.g. it has `Read` on a document and wants `Write`; it has a directory with `Enumerate` and wants `Create`). Requires `Grant` on `presence`. If `forProcess` is set, the caller must hold `Manage` on it and the process must be in the caller's job tree. Completion:

`result0` = `Handle<Node>` (or `0` when `forProcess` was set — the Node went to the child), `result1` = `requestId`, `aux` = audit reference. Errors: `NotMainProcess`, `NoInteraction`, `NotFrontmost`, `UserDenied`, `InvalidArgument` (rights not a superset of current), `GrantExpired/Revoked`.

```
Picker.present(presence: Handle<Presence>, mode: PickerMode, options:
PickerOptions, token: InteractionToken, port: Handle<Port>, key: Key) ->
Result<Void, Error>
C  tessera_error_t tessera_picker_present(tessera_handle_t presence, uint32_t
mode, const tessera_picker_options_t* options, const
tessera_interaction_token_t* token, tessera_handle_t port, uint64_t key);
```

Shows the trusted Tessera picker. The user's selection *is* the grant: the completion's `result0` is a Node for the chosen file/directory (single modes) or a `NodeKind.Selection` virtual directory whose entries are the chosen items (multi modes; `lookup` each by name to get its Node — each carries the granted rights), with no access to anything around them. Save modes return a Node with `Create` on the chosen directory restricted to the chosen name (`Node.create` with any other name fails `AccessDenied`) — or, with `SaveFile`, the created file's Node directly. `result1` = `requestId`, `aux` = audit reference. With `forProcess` set the result is delivered to the child as `grantDelivered` and `result0 = 0`. Errors as above plus `Cancelled` if the user dismissed the picker.

```
Grant.restore(presence: Handle<Presence>, grantId: GrantId, port: Handle<Port>,
key: Key) -> Result<Void, Error>
C  tessera_error_t tessera_grant_restore(tessera_handle_t presence, const
uint8_t grant_id[16], tessera_handle_t port, uint64_t key);
```

Re-obtains a `Persistent` grant's Node in a later session with no user interaction (no token needed — the user already decided). `result0` = `Handle<Node>`. Errors: `NotFound`, `GrantExpired`, `GrantRevoked` (the user withdrew it in Settings; the app should forget the id).

```
Grant.release(grantId: GrantId, port: Handle<Port>, key: Key) -> Result<Void,
Error>
C  tessera_error_t tessera_grant_release(const uint8_t grant_id[16],
tessera_handle_t port, uint64_t key);
```

Voluntarily ends a grant (any process holding a Node from it may call this). Nodes from the grant fail `GrantRevoked` afterwards.

```
Grant.list(presence: Handle<Presence>, out: Buffer, port: Handle<Port>, key:
Key) -> Result<Void, Error>
Grant.info(grantId: GrantId, out: Buffer, port: Handle<Port>, key: Key) ->
Result<Void, Error>
C  tessera_error_t tessera_grant_list(tessera_handle_t presence,
tessera_buffer_t out, tessera_handle_t port, uint64_t key);
C  tessera_error_t tessera_grant_info(const uint8_t grant_id[16],
tessera_buffer_t out, tessera_handle_t port, uint64_t key);
```

`list` returns `Vector<GrantRecord>` for the app; `info` one record. `result0` = bytes written; `BufferTooSmall` with the size in `detail`.

14.4 Other interactive authority

The same presence + token discipline covers every runtime prompt, so no domain invents its own:

```
Authorization.request(presence: Handle<Presence>, capability: Buffer,
parameters: Buffer, forProcess: Handle<Process>?, token: InteractionToken,
port: Handle<Port>, key: Key) -> Result<Void, Error>
C  tessera_error_t tessera_authorization_request(tessera_handle_t presence,
tessera_buffer_t capability, tessera_buffer_t parameters, tessera_handle_t
for_process, const tessera_interaction_token_t* token, tessera_handle_t port,
uint64_t key);
```

`capability` is a dotted name (`tessera.agent.control`, `tessera.location.precise`, `tessera.contacts.read`, `tessera.camera`, …); `parameters` is a §10.3-encoded struct defined by that capability. On approval the entitlement page is updated (`entitlementChanged`) and, for handle-bearing capabilities, `result0` is the minted handle (e.g. a `Channel` to `tessera.agent`). `result1` = `requestId`, `aux` = audit reference.

```
Settings.redirect(presence: Handle<Presence>, setting: Buffer, token:
InteractionToken, port: Handle<Port>, key: Key) -> Result<Void, Error>
C  tessera_error_t tessera_settings_redirect(tessera_handle_t presence,
tessera_buffer_t setting, const tessera_interaction_token_t* token,
tessera_handle_t port, uint64_t key);
```

Apps can never change time, locale, or any system setting. This is the one sanctioned move: open Settings on `setting` (or, for settings that allow it, a per-setting OS prompt). Completion `result0` = 1 if the user changed it, 0 otherwise.

14.5 The child-brokering flow, end to end

1. The user types a command in a visible terminal. MosaicDisplay delivers the key events to the terminal's surface, each carrying an `InteractionToken`. The terminal's runtime keeps the most recent token per surface.
2. The terminal launches the CLI tool with `Process.launch`, passing `ContainerTemp`, `StdIn/Out/Err`, and a `ParentChannel`. The tool gets its own `System` channel from the Launcher (`isMain: false`, no Presence).
3. The tool needs `~/Projects/foo`. It has no Node for it. It sends a request on `ParentChannel` (a protocol the terminal defines; the spec only requires the transport).
4. The terminal (main process) calls `Picker.present(presence, .OpenDirectory, options{ rights: Read|Enumerate|Traverse|Write|Create, lifetime: .Session, forProcess: toolProcess }, token)`.
5. The Presence service validates the token with MosaicDisplay, checks the terminal's surface is front-facing, shows the trusted picker.
6. The user picks the directory. The Namespace service adds the Node to the *tool's* view; the tool receives `grantDelivered(requestId, node, grantId)` on its System channel; the terminal's completion carries `requestId` and `result0 = 0`.
7. The tool proceeds. When it exits, `Session` grants issued for it are released; the Audit log has the whole chain.

An IDE running a compiler is the same flow with "compiler" for "tool". A hidden tab, background task or daemon has no front-facing surface, therefore no valid token, therefore no prompt.

14.6 Backend

The Presence service is a Tessera process; MosaicDisplay is the token authority. LibTessera's role is only transport (`Channel.call` on `tessera.presence`) and the handle types. Nothing here depends on the kernel.

---

15. Network

15.1 Model

An `Endpoint` is a network endpoint of a stated family and protocol. LibTessera offers exactly the kernel-level socket surface — bind, listen, accept, connect, send, receive, options — as completion-based operations on typed handles. Name resolution (`tessera.resolver`), interface/connectivity state (`tessera.network`), proxies and TLS are services or libraries above. Local IPC uses channels, never sockets.

Entitlements are enforced by the sandbox at process creation (§21): `tessera.network.client` (outbound), `tessera.network.server` (bind non-ephemeral, listen), `tessera.network.local` (loopback only), `tessera.network.raw` (raw/ICMP; system only). No entitlement → `Endpoint.create` fails `NotEntitled` and the kernel refuses the underlying object too.

15.2 Types

```
enum AddressFamily: UInt32   Ipv4 = 0, Ipv6 = 1
enum Protocol: UInt32        Tcp = 0, Udp = 1, Raw = 2, Icmp = 3

struct NetAddress (32 bytes)
  family: u32 @0
  port:   u16 @4
  flags:  u16 @6      bit0 SCOPE_VALID
  bytes:  [u8;16] @8  IPv4 in the first 4 bytes
  scope:  u32 @24     IPv6 scope/interface id
  reserved: u32 @28

enum EndpointOptions: UInt32 (flags)   None = 0, ReuseAddress = 1<<0, DualStack
= 1<<1 (Ipv6 endpoint also accepts Ipv4-mapped)
enum SendFlags: UInt32 (flags)         None = 0, NoSignal = 0 (there are no
signals; reserved), Push = 1<<0, More = 1<<1 (cork)
enum ReceiveFlags: UInt32 (flags)      None = 0, Peek = 1<<0, WaitAll = 1<<1
enum ShutdownHow: UInt32               Read = 0, Write = 1, Both = 2

enum EndpointOption: UInt32
  NoDelay = 0, KeepAlive = 1, KeepAliveIdle = 2, KeepAliveInterval = 3,
KeepAliveCount = 4,
  ReceiveBuffer = 5, SendBuffer = 6, Ttl = 7, HopLimit = 8, Broadcast = 9,
MulticastJoin = 10, MulticastLeave = 11,
  MulticastTtl = 12, MulticastLoop = 13, Linger = 14, ReceiveTimestamps = 15,
TcpUserTimeout = 16, TypeOfService = 17,
  BindToInterface = 18, ReceiveErrorQueue = 19, TcpFastOpen = 20, IpV6Only = 21
```

```
struct EndpointInfo (topic Endpoint, v1, 96 bytes)
  family: u32 @0, protocol: u32 @4, state: u32 @8 (Unbound=0, Bound=1,
Listening=2, Connecting=3, Connected=4, Closed=5),
  reserved0: u32 @12, local: NetAddress @16, peer: NetAddress @48, bytesSent:
u64 @80, bytesReceived: u64 @88
```

15.3 Calls

```
Endpoint.create(family: AddressFamily, protocol: Protocol, options:
EndpointOptions) -> Result<Handle<Endpoint>, Error>
C  tessera_error_t tessera_endpoint_create(uint32_t family, uint32_t protocol,
uint32_t options, tessera_handle_t* out);
```

Synchronous. Errors: `NotEntitled`, `Unsupported`, `QuotaExceeded`.

```
Endpoint.bind(endpoint: Handle<Endpoint>, address: NetAddress) -> Result<Void,
Error>
Endpoint.listen(endpoint: Handle<Endpoint>, backlog: UInt32) -> Result<Void,
Error>
C  tessera_error_t tessera_endpoint_bind(tessera_handle_t endpoint, const
tessera_net_address_t* address);
C  tessera_error_t tessera_endpoint_listen(tessera_handle_t endpoint, uint32_t
backlog);
```

Synchronous. Errors: `AddressInUse`, `AddressNotAvailable`, `NotEntitled` (server), `BadState`.

```
Endpoint.accept(endpoint: Handle<Endpoint>, peer: NetAddress*, port:
Handle<Port>, key: Key) -> Result<Void, Error>
C  tessera_error_t tessera_endpoint_accept(tessera_handle_t endpoint,
tessera_net_address_t* out_peer, tessera_handle_t port, uint64_t key);
```

Completion kind `Net`: `result0` = new `Handle<Endpoint>`; `out_peer` filled before the packet is posted. Errors: `BadState`, `ConnectionReset` (the pending connection died), `QuotaExceeded`.

```
Endpoint.connect(endpoint: Handle<Endpoint>, address: NetAddress, port:
Handle<Port>, key: Key) -> Result<Void, Error>
C  tessera_error_t tessera_endpoint_connect(tessera_handle_t endpoint, const
tessera_net_address_t* address, tessera_handle_t port, uint64_t key);
```

For UDP, sets the default peer. Errors: `ConnectionRefused`, `Unreachable`, `Timeout` (cancel to impose a deadline), `AddressNotAvailable`.

```
Endpoint.send(endpoint: Handle<Endpoint>, buffers: [Buffer], bufferCount:
UInt32, flags: SendFlags, port: Handle<Port>, key: Key) -> Result<Void, Error>
Endpoint.receive(endpoint: Handle<Endpoint>, buffers: [Buffer], bufferCount:
UInt32, flags: ReceiveFlags, port: Handle<Port>, key: Key) -> Result<Void,
Error>
C  tessera_error_t tessera_endpoint_send(tessera_handle_t endpoint, const
tessera_buffer_t* buffers, uint32_t buffer_count, uint32_t flags,
tessera_handle_t port, uint64_t key);
C  tessera_error_t tessera_endpoint_receive(tessera_handle_t endpoint, const
tessera_buffer_t* buffers, uint32_t buffer_count, uint32_t flags,
tessera_handle_t port, uint64_t key);
```

`result0` = bytes; `result1` = `ReceiveTimestamps` timestamp if enabled; `aux` = flags (bit0 TRUNCATED for datagrams). 0 bytes received on a stream = orderly close. Errors: `ConnectionReset`, `PeerClosed`, `Unreachable`.

```
Endpoint.sendTo(endpoint, buffers, bufferCount, address: NetAddress, flags,
port, key) -> Result<Void, Error>
Endpoint.receiveFrom(endpoint, buffers, bufferCount, source: NetAddress*,
flags, port, key) -> Result<Void, Error>
C  tessera_error_t tessera_endpoint_send_to(tessera_handle_t endpoint, const
tessera_buffer_t* buffers, uint32_t buffer_count, const tessera_net_address_t*
address, uint32_t flags, tessera_handle_t port, uint64_t key);
C  tessera_error_t tessera_endpoint_receive_from(tessera_handle_t endpoint,
const tessera_buffer_t* buffers, uint32_t buffer_count, tessera_net_address_t*
out_source, uint32_t flags, tessera_handle_t port, uint64_t key);
```

```
Endpoint.sendMany(endpoint, messages: [Datagram], count: UInt32, flags, port,
key) -> Result<Void, Error>
Endpoint.receiveMany(endpoint, messages: [Datagram], count: UInt32, flags,
port, key) -> Result<Void, Error>
struct Datagram (56 bytes)   buffers: Address @0 (pointer to [Buffer]),
bufferCount: u32 @8, flags: u32 @12 (out: TRUNCATED), address: NetAddress @16,
transferred: u64 @48 (out)
C  tessera_error_t tessera_endpoint_send_many(tessera_handle_t endpoint,
tessera_datagram_t* messages, uint32_t count, uint32_t flags, tessera_handle_t
port, uint64_t key);
C  tessera_error_t tessera_endpoint_receive_many(tessera_handle_t endpoint,
tessera_datagram_t* messages, uint32_t count, uint32_t flags, tessera_handle_t
port, uint64_t key);
```

Batched datagram I/O (QUIC needs it). `result0` = messages completed.

```
Endpoint.shutdown(endpoint: Handle<Endpoint>, how: ShutdownHow) -> Result<Void,
Error>
Endpoint.setOption(endpoint: Handle<Endpoint>, option: EndpointOption, value:
Buffer) -> Result<Void, Error>
Endpoint.getOption(endpoint: Handle<Endpoint>, option: EndpointOption, out:
Buffer) -> Result<Size, Error>
C  tessera_error_t tessera_endpoint_shutdown(tessera_handle_t endpoint,
uint32_t how);
C  tessera_error_t tessera_endpoint_set_option(tessera_handle_t endpoint,
uint32_t option, tessera_buffer_t value);
C  tessera_error_t tessera_endpoint_get_option(tessera_handle_t endpoint,
uint32_t option, tessera_buffer_t out, uint64_t* out_size);
```

Synchronous. Option values are `u32` unless the option is an address (`MulticastJoin/Leave`: `NetAddress` + interface) or a string (`BindToInterface`). `Unsupported` for options the backend lacks.

Readiness-style consumers use `Object.waitAsync(endpoint, READABLE|WRITABLE|…)`; `Object.info(endpoint, .Endpoint)` gives addresses and state.

15.4 Backend

Linux: `socket` / `bind` / `listen` direct (allowed by the entitlement-derived seccomp filter), `ACCEPT` / `CONNECT` / `SENDMSG` / `RECVMSG` / `SHUTDOWN` as io_uring SQEs, `setsockopt` / `getsockopt` direct; `sendMany` / `receiveMany` = `sendmmsg` / `recvmmsg`. XNU: BSD sockets with a worker pool completing via the Port's kqueue. A native kernel: endpoints are objects.

---

16. Device streams

16.1 Model

Every continuous hardware data path — sensors, audio in/out, camera frames, haptic patterns — is a `Stream`: a ring of `DeviceShareable` `Memory` buffers shared between the process and the Devices service (which owns the hardware), with buffer ownership passed back and forth as completions. Output streams: the app enqueues a filled buffer, the completion says it was consumed. Input streams: the app enqueues an empty buffer, the completion says it was filled and carries

timestamps. This one model serves 3-axis accelerometer samples at 200 Hz, 48 kHz audio, and 4K camera frames; only the descriptor differs.

Privacy: the Devices service turns on the system indicator for microphone/camera, refuses capture from backgrounded processes unless entitled (`tessera.audio.capture.background`), and every open is audited.

16.2 Types

```
enum DeviceClass: UInt32   Sensor = 0, AudioInput = 1, AudioOutput = 2, Camera
= 3, Haptic = 4
enum SensorKind: UInt32
  Accelerometer = 0, Gyroscope = 1, Magnetometer = 2, Barometer = 3,
AmbientLight = 4, Proximity = 5,
  Gravity = 6, LinearAcceleration = 7, RotationVector = 8, StepCounter = 9,
StepDetector = 10,
  Temperature = 11, Humidity = 12, HeartRate = 13, Hinge = 14 (fold angle),
Orientation = 15, Significant = 16 (motion trigger)

struct DeviceInfo (variable, §10.3)   deviceId: UInt64, class: DeviceClass,
subKind: UInt32 (SensorKind / audio role / camera facing),
  name: String, vendor: String, capabilities: Bytes (class-specific), flags:
UInt32 (bit0 DEFAULT, bit1 REMOVABLE, bit2 WAKEUP_CAPABLE)

struct StreamDescriptor (96 bytes)
  class:        u32 @0
  deviceId:     u64 @8      0 = default device of the class
  bufferCount:  u32 @16     2–32
  bufferSize:   u32 @20     bytes per buffer; 0 = computed from the format
  flags:        u32 @24     bit0 LOW_LATENCY, bit1 WAKEUP (sensor may wake the
CPU), bit2 EXCLUSIVE
  reserved0:    u32 @28
  format:       StreamFormat @32 (64 bytes, union by class)

union StreamFormat (64 bytes)
  audio:  { sampleRate: u32, channels: u32, sampleFormat: u32 (I16=0, I24=1,
I32=2, F32=3), framesPerBuffer: u32, channelMask: u64, reserved: [u64;5] }
  camera: { pixelFormat: u32 (fourcc), width: u32, height: u32,
framesPerSecondQ16: u32, colorSpace: u32, hdrMode: u32, reserved: [u64;5] }
  sensor: { rateHz: u32, maxReportLatencyNs: u64 (batching), sampleFormat: u32
(Vector3F32=0, ScalarF32=1, QuaternionF32=2, U64Count=3), reserved: [u64;5] }
  haptic: { patternFormat: u32 (AmplitudeF32AtHz=0), rateHz: u32, reserved:
[u64;7] }

struct StreamPacket (32 bytes)   metadata travelling with a buffer
  timestamp: Instant @0     Boottime capture/playback time
  length:    u32 @8         valid bytes
  flags:     u32 @12        bit0 DISCONTINUITY, bit1 END_OF_STREAM, bit2
```

```
KEYFRAME (camera compressed), bit3 SILENCE
  sequence:  u64 @16
  reserved:  u64 @24

struct StreamInfo (topic Stream, v1, 64 bytes)
  state: u32 @0 (Configured=0, Running=1, Stopped=2, Error=3), latency:
Duration @8, dropped: u64 @16,
  delivered: u64 @24, underruns: u64 @32, descriptor: StreamDescriptor
(partial: class/deviceId/bufferCount/bufferSize) @40 (24 bytes)
```

16.3 Calls

```
Device.list(class: DeviceClass, out: Buffer, port: Handle<Port>, key: Key) ->
Result<Void, Error>
C  tessera_error_t tessera_device_list(uint32_t class, tessera_buffer_t out,
tessera_handle_t port, uint64_t key);
```

`Vector<DeviceInfo>`; `result0` = bytes. Requires the class entitlement (`tessera.sensors.motion` covers Accelerometer/Gyroscope/Gravity/LinearAcceleration/RotationVector/StepCounter/StepDetector/Significant; `tessera.sensors.environment` the rest; `tessera.audio.capture`; `tessera.audio.playback`; `tessera.camera`; `tessera.haptics`). Hot-plug arrives as `deviceChanged` on the System channel.

```
Stream.open(descriptor: StreamDescriptor, port: Handle<Port>, key: Key) ->
Result<Void, Error>
C  tessera_error_t tessera_stream_open(const tessera_stream_descriptor_t*
descriptor, tessera_handle_t port, uint64_t key);
```

Completion kind `Stream`: `result0` = `Handle<Stream>`, `result1` = negotiated buffer size, `aux` = negotiated buffer count. The service may adjust the format within what it advertised; read back with `Object.info(.Stream)`. Errors: `NotEntitled`, `NotFound`, `FormatUnsupported`, `DeviceBusy` (EXCLUSIVE held by another), `UserDenied` (the class needs a one-time authorization the app never obtained — obtain it with `Authorization.request`).

```
Stream.buffer(stream: Handle<Stream>, index: UInt32) -> Result<Handle<Memory>,
Error>
C  tessera_error_t tessera_stream_buffer(tessera_handle_t stream, uint32_t
index, tessera_handle_t* out);
```

Synchronous. Returns the `Memory` for ring slot `index` (mappable; `Read` for input streams, `Read|Write` for output). Call once per slot and map.

```
Stream.enqueue(stream: Handle<Stream>, index: UInt32, packet: StreamPacket,
port: Handle<Port>, key: Key) -> Result<Void, Error>
C  tessera_error_t tessera_stream_enqueue(tessera_handle_t stream, uint32_t
index, const tessera_stream_packet_t* packet, tessera_handle_t port, uint64_t
key);
```

Hands slot `index` to the device. Completion when the device is done with it: `result0` = `index`, `result1` = `packet.timestamp` (actual), `aux` = `packet.length` | (`packet.flags` << 32). For output streams `packet` describes the data the app wrote; for input streams the app passes an empty packet and reads the completion's. `Underrun` / `Overrun` are reported in the completion `error` with the slot still returned. Errors: `BadState`, `OutOfRange`, `DeviceLost`.

```
Stream.start(stream: Handle<Stream>, port: Handle<Port>, key: Key) ->
Result<Void, Error>
Stream.stop(stream: Handle<Stream>, port: Handle<Port>, key: Key) ->
Result<Void, Error>
Stream.flush(stream: Handle<Stream>, port: Handle<Port>, key: Key) ->
Result<Void, Error>
C  tessera_error_t tessera_stream_start(tessera_handle_t stream,
tessera_handle_t port, uint64_t key);
C  tessera_error_t tessera_stream_stop(tessera_handle_t stream,
tessera_handle_t port, uint64_t key);
C  tessera_error_t tessera_stream_flush(tessera_handle_t stream,
tessera_handle_t port, uint64_t key);
```

`stop` returns every enqueued slot with `Cancelled`. `flush` (output) completes when everything enqueued has actually played/been consumed.

```
Stream.reconfigure(stream: Handle<Stream>, descriptor: StreamDescriptor, port:
Handle<Port>, key: Key) -> Result<Void, Error>
C  tessera_error_t tessera_stream_reconfigure(tessera_handle_t stream, const
tessera_stream_descriptor_t* descriptor, tessera_handle_t port, uint64_t key);
```

Only while stopped; buffers are reallocated and `Stream.buffer` must be called again.

```
Stream.control(stream: Handle<Stream>, request: UInt32, in: Buffer, out:
Buffer, port: Handle<Port>, key: Key) -> Result<Void, Error>
C  tessera_error_t tessera_stream_control(tessera_handle_t stream, uint32_t
```

```
request, tessera_buffer_t in, tessera_buffer_t out, tessera_handle_t port,
uint64_t key);
```

Class-specific controls with stable ids published per class (camera: focus, exposure, zoom, torch; audio: volume/route hints; haptic: intensity). Unknown request → `Unsupported`. This is the only extension point and it is typed by class, not an ioctl free-for-all.

16.4 Audio specifics

Audio output apps request `LOW_LATENCY` with `framesPerBuffer` at the device's minimum (reported in `DeviceInfo.capabilities`), run a thread with `Thread.setRealtime` sized to the buffer period, and keep ≥ 2 slots enqueued. The Devices service mixes (system mixer) unless `EXCLUSIVE` is granted (`tessera.audio.exclusive`, media platforms). Routing (which speaker, Bluetooth) is system policy exposed via `tessera.devices` events, never a per-app choice beyond `Stream.control` hints.

16.5 Backend

Linux: the Devices service owns ALSA (`snd_pcm` via direct ioctls, no alsa-lib), V4L2 (`VIDIOC_*` with `DMABUF` memory), IIO (`iio_buffer` character devices) and the haptic input FF interface; ring buffers are dma-buf/udmabuf `Memory`, completions travel over a per-stream channel that LibTessera folds into the Port. XNU: CoreAudio HAL / AVFoundation / CoreMotion equivalents behind the same service. A stream's kernel-facing implementation is entirely inside the service; apps never see a device node.

---

17. GPU and fences

17.1 Model

LibTessera does not expose a graphics API. It exposes: GPU devices as objects, device-shareable buffers, fences for cross-process/cross-device synchronisation, and a typed control escape hatch that only system components (`tessera.driver`) may use — Kira Graphics' backend, the Mesa compatibility shim, MosaicDisplay. Ordinary apps allocate buffers and fences, hand them to Kira Graphics (in-

process) or to MosaicDisplay (over `tessera.display`), and never issue GPU commands through LibTessera.

17.2 Types

```
struct GpuInfo (variable, §10.3)   deviceId: UInt64, name: String, vendorId:
UInt32, deviceIdRaw: UInt32, driver: String,
  flags: UInt32 (bit0 INTEGRATED, bit1 DEFAULT, bit2 DISPLAY_ATTACHED),
memoryBytes: UInt64, capabilities: Bytes

struct GpuBufferDescriptor (48 bytes)
  size:      u64 @0
  usage:     u32 @8     bit0 RENDER_TARGET, bit1 TEXTURE, bit2 VERTEX, bit3
UNIFORM, bit4 STORAGE, bit5 SCANOUT (compositor may scan out), bit6 CPU_READ,
bit7 CPU_WRITE, bit8 PROTECTED (DRM content)
  format:    u32 @12    fourcc; 0 = linear blob
  width:     u32 @16
  height:    u32 @20
  modifier:  u64 @24    layout modifier (0 = let the driver pick; read back
from GpuBufferInfo)
  alignment: u32 @32
  flags:     u32 @36    bit0 PREFER_DEVICE_LOCAL
  reserved:  u64 @40

struct GpuBufferInfo (topic Memory, v2, adds to MemoryInfo)   planeCount: u32,
planes: [{ offset: u64, stride: u32, size: u32 }; 4], modifier: u64, format:
u32, width: u32, height: u32

enum FenceKind: UInt32   Binary = 0, Timeline = 1
struct FenceInfo (topic Fence, v1, 24 bytes)   kind: u32 @0, signaled: u8 @4,
reserved: [u8;3] @5, value: u64 @8 (timeline), signaledAt: Instant @16
```

17.3 Calls

```
Gpu.list(out: Buffer, port: Handle<Port>, key: Key) -> Result<Void, Error>
C  tessera_error_t tessera_gpu_list(tessera_buffer_t out, tessera_handle_t
port, uint64_t key);
```

`Vector<GpuInfo>`.

```
Gpu.open(deviceId: UInt64, port: Handle<Port>, key: Key) -> Result<Void, Error>
C  tessera_error_t tessera_gpu_open(uint64_t device_id, tessera_handle_t port,
uint64_t key);
```

`result0` = `Handle<GpuDevice>` (rights `Read | Write | Inspect | Duplicate`; `Manage` only with `tessera.driver`). `deviceId = 0` = default. Entitlement `tessera.gpu` (default for every app).

```
Gpu.allocate(device: Handle<GpuDevice>, descriptor: GpuBufferDescriptor, port:
Handle<Port>, key: Key) -> Result<Void, Error>
C  tessera_error_t tessera_gpu_allocate(tessera_handle_t device, const
tessera_gpu_buffer_descriptor_t* descriptor, tessera_handle_t port, uint64_t
key);
```

`result0` = `Handle<Memory>` (DeviceShareable, mappable if CPU_READ/WRITE), `result1` = chosen modifier. Errors: `OutOfMemory`, `FormatUnsupported`, `NotEntitled` (PROTECTED, SCANOUT without display entitlement).

```
Gpu.import(device: Handle<GpuDevice>, memory: Handle<Memory>, port:
Handle<Port>, key: Key) -> Result<Void, Error>
C  tessera_error_t tessera_gpu_import(tessera_handle_t device, tessera_handle_t
memory, tessera_handle_t port, uint64_t key);
```

Makes a DeviceShareable buffer from another device or a stream (camera frame) usable on this device. `result0` = the same `memory` handle re-tagged with device residency info (no new handle); `Unsupported` if the modifier is foreign to the device.

```
Fence.create(kind: FenceKind, initialValue: UInt64) -> Result<Handle<Fence>,
Error>
Fence.signal(fence: Handle<Fence>, value: UInt64) -> Result<Void, Error>
Fence.waitAsync(fence: Handle<Fence>, value: UInt64, port: Handle<Port>, key:
Key) -> Result<Void, Error>
Fence.merge(fences: [Handle<Fence>], count: UInt32) -> Result<Handle<Fence>,
Error>
C  tessera_error_t tessera_fence_create(uint32_t kind, uint64_t initial_value,
tessera_handle_t* out);
C  tessera_error_t tessera_fence_signal(tessera_handle_t fence, uint64_t
value);
C  tessera_error_t tessera_fence_wait_async(tessera_handle_t fence, uint64_t
value, tessera_handle_t port, uint64_t key);
C  tessera_error_t tessera_fence_merge(const tessera_handle_t* fences, uint32_t
count, tessera_handle_t* out);
```

Binary fences signal once (`value` ignored); timeline fences signal monotonically increasing values, and `waitAsync(value)` completes when the fence reaches ≥ `value`. Fences carry `Transfer` — they cross to MosaicDisplay with the buffer they guard. `merge` produces a binary fence that signals when all inputs have.

`Object.waitAsync(fence, SIGNALED)` is equivalent to `waitAsync(fence, 1)` for binary fences.

```
Gpu.control(device: Handle<GpuDevice>, request: UInt32, in: Buffer, out:
Buffer, handlesIn: [Handle], handlesInCount: UInt32, handlesOut: [Handle],
handlesOutCapacity: UInt32, port: Handle<Port>, key: Key) -> Result<Void,
Error>
C  tessera_error_t tessera_gpu_control(tessera_handle_t device, uint32_t
request, tessera_buffer_t in, tessera_buffer_t out, const tessera_handle_t*
handles_in, uint32_t handles_in_count, tessera_handle_t* handles_out, uint32_t
handles_out_capacity, tessera_handle_t port, uint64_t key);
```

Requires `Manage` and `tessera.driver`. Request ids are namespaced per backend (`0x4C58_xxxx` Linux DRM pass-through, `0x584E_xxxx` XNU IOKit, `0x5453_xxxx` Tessera-native driver protocol). This is the *one* deliberately kernel-specific surface, fenced off so that only the driver stack (a system component that is rewritten per kernel anyway) touches it. Apps that call it fail `NotEntitled`.

17.4 Display handoff (for reference; protocol owned by MosaicDisplay)

An app's KCoreSurface presents by sending `{ buffer: Handle<Memory>, acquireFence: Handle<Fence>, damage, transform }` over `tessera.display`; MosaicDisplay returns a release fence. The buffer must have `SCANOUT` when the compositor wants to scan it out directly. Nothing in LibTessera knows about surfaces; it only carries handles.

17.5 Backend

Linux: `GpuDevice` = DRM render node fd (`/dev/dri/renderD*`, opened by the Devices service and passed in); `Gpu.allocate` = GBM/driver-specific allocation exported as dma-buf (`Memory` with `DeviceShareable`); `Fence` binary = `sync_file`, timeline = `drm_syncobj`; `Gpu.control` = `ioctl` pass-through for entitled system components. XNU: `MTLDevice`/IOAccelerator, `IOSurface`, `MTLSharedEvent`.

---

18. Power and thermal

18.1 Types

```
struct PowerInfo (64 bytes)   read from a system-maintained shared page; no
trap
  batteryPercent:  u32 @0     0–100, 0xFFFF_FFFF = no battery
  batteryState:    u32 @4     Unknown=0, Discharging=1, Charging=2, Full=3,
NotCharging=4
  externalPower:   u32 @8     None=0, Usb=1, Wireless=2, Dock=3, Mains=4
  thermalLevel:    u32 @12    Nominal=0, Fair=1, Serious=2, Critical=3
  lowPowerMode:    u8  @16
  reserved0:       [u8;7] @17
  timeToEmpty:     Duration @24  0 = unknown
  timeToFull:      Duration @32
  powerDrawMilliwatts: u32 @40
  cpuThrottlePercent:  u32 @44   0 = none
  gpuThrottlePercent:  u32 @48
  reserved1:       u32 @52
  updated:         Instant @56   Boottime

enum WakeLockKind: UInt32   Cpu = 0 (keep running while backgrounded), Display
= 1 (keep the screen on; GUI front-most only), Audio = 2 (implied by an active
audio output stream), Network = 3 (keep radios up for a transfer)
```

18.2 Calls

```
Power.read() -> Result<PowerInfo, Error>
C  tessera_error_t tessera_power_read(tessera_power_info_t* out);
```

Synchronous, reads the shared page. Changes are pushed as `powerChanged` / `thermalChanged` events on the System channel.

```
WakeLock.acquire(kind: WakeLockKind, reason: Buffer, timeout: Duration, port:
Handle<Port>, key: Key) -> Result<Void, Error>
C  tessera_error_t tessera_wakelock_acquire(uint32_t kind, tessera_buffer_t
reason, uint64_t timeout, tessera_handle_t port, uint64_t key);
```

Entitlement `tessera.power.wakelock` (`Display` needs only front-most; `Audio` is automatic). `result0` = `Handle<WakeLock>`; closing releases. `timeout` caps the lock (0 = system default, never unlimited for `Cpu`). The system may revoke under

thermal or battery policy: the handle asserts `PEER_CLOSED` and `Object.info` shows `revoked`. Every acquisition is audited and shown in Settings' battery view.

```
Power.scheduleWake(deadline: Instant, tolerance: Duration, port: Handle<Port>,
key: Key) -> Result<Void, Error>
C  tessera_error_t tessera_power_schedule_wake(uint64_t deadline, uint64_t
tolerance, tessera_handle_t port, uint64_t key);
```

Entitlement `tessera.power.wake`. Asks to have the process resumed (or launched with `LaunchInfo.reason = Resume`) at `deadline` (Boottime) ± `tolerance`. The completion is the wake itself. Equivalent to a `Timer.set(.Wake)` that survives process death.

Backend: Linux `wakeup_source` via the Power service (`/sys/power/wake_lock` on Android-style kernels, or `epoll` on a wake-capable timerfd), `CLOCK_BOOTTIME_ALARM`; thermal via `thermal_zone`; battery via `power_supply` uevents — all inside the Power service, which publishes the shared page. XNU: IOPMAssertions, `IOPSCopyPowerSourcesInfo`.

---

19. Random

```
Random.fill(out: Buffer) -> Result<Void, Error>
C  tessera_error_t tessera_random_fill(tessera_buffer_t out);
```

Cryptographically secure bytes from the kernel CSPRNG; never blocks after early boot (LibTessera's init on a certified device asserts the pool is seeded). This is the whole surface: key derivation, hashing and ciphers are libraries above (Kira's crypto framework), not system calls.

Backend: Linux `getrandom(0)`, XNU `getentropy` / `SecRandomCopyBytes` equivalent syscall.

---

20. Diagnostics and system information

20.1 Logging and tracing

```
enum LogLevel: UInt32   Trace = 0, Debug = 1, Info = 2, Notice = 3, Warning =
4, Error = 5, Fault = 6

Log.write(level: LogLevel, subsystem: Buffer, category: Buffer, message:
Buffer) -> Result<Void, Error>
C  tessera_error_t tessera_log_write(uint32_t level, tessera_buffer_t
subsystem, tessera_buffer_t category, tessera_buffer_t message);
```

Structured, ring-buffered by the system logger; `subsystem`/`category` ≤ 64 bytes, `message` ≤ 4 KiB. Privacy: messages are private by default in the persisted log (redacted for other apps and for non-debug images); `Fault` entries trigger a crash-report-style capture.

```
Trace.enabled(category: Buffer) -> Bool
Trace.begin(category: Buffer, name: Buffer, args: Buffer) -> Void
Trace.end(category: Buffer) -> Void
Trace.instant(category: Buffer, name: Buffer, args: Buffer) -> Void
Trace.counter(category: Buffer, name: Buffer, value: Int64) -> Void
Trace.flow(category: Buffer, name: Buffer, flowId: UInt64, phase: UInt32) ->
Void    (phase: Begin=0, Step=1, End=2)
C  uint8_t tessera_trace_enabled(tessera_buffer_t category);
C  void tessera_trace_begin(tessera_buffer_t category, tessera_buffer_t name,
tessera_buffer_t args);
C  void tessera_trace_end(tessera_buffer_t category);
C  void tessera_trace_instant(tessera_buffer_t category, tessera_buffer_t name,
tessera_buffer_t args);
C  void tessera_trace_counter(tessera_buffer_t category, tessera_buffer_t name,
int64_t value);
C  void tessera_trace_flow(tessera_buffer_t category, tessera_buffer_t name,
uint64_t flow_id, uint32_t phase);
```

Zero-cost when the category is disabled (a shared-page bitmap check). Trace events go to a per-process shared ring consumed by the system tracer; `args` is §10.3-encoded key/value pairs. Entitlement `tessera.trace` (default).

20.2 Crash and debug (debug image / entitled only)

```
Crash.setReporter(channel: Handle<Channel>) -> Result<Void, Error>
C  tessera_error_t tessera_crash_set_reporter(tessera_handle_t channel);
```

Registers a channel that receives `CrashReport { thread: u64, reason: u32, faultAddress: u64, registers: Bytes, backtrace: Vector<u64> }` for any fatal fault in this process, produced by the system (not by the crashing process) before termination. Entitlement `tessera.debug.crash`. Production apps get crash reports through the Store/Settings pipeline instead.

```
Debug.break() -> Void
Debug.isAttached() -> Bool
C  void tessera_debug_break(void);
C  uint8_t tessera_debug_is_attached(void);
```

`break` traps into an attached debugger, or is a no-op on production images.

20.3 System information

```
enum InfoTopic (for System.info): UInt32   Version = 0, Machine = 1, Backend =
2, Device = 3, Limits = 4, Session = 5

System.info(topic: InfoTopic, version: UInt32, out: Buffer) -> Result<Size,
Error>
C  tessera_error_t tessera_system_info(uint32_t topic, uint32_t version,
tessera_buffer_t out, uint64_t* out_written);

struct VersionInfo (v1, 32 bytes)   osCalVer: u32 @0 (YYYYMMW), libTesseraAbi:
u32 @4, buildId: u64 @8, flags: u32 @16 (bit0 DEBUG_IMAGE, bit1 CERTIFIED),
reserved: [u8;12] @20
struct MachineInfo (v1, 64 bytes)   arch: u32 @0 (Arm64=0, X64=1, RiscV64=2),
pageSize: u32 @4, cpuCount: u32 @8, performanceCpuCount: u32 @12, memoryTotal:
u64 @16, cacheLineSize: u32 @24, reserved0: u32 @28, features: u64 @32 (arch
feature bits), model: u64 @40 (hash), reserved1: [u64;2] @48
struct BackendInfo (v1, 32 bytes)   kind: u32 @0 (Linux=0, Xnu=1, Native=2),
major: u32 @4, minor: u32 @8, patch: u32 @12, capabilities: u64 @16 (bit0
KERNEL_RIGHTS: handle rights are kernel-enforced), reserved: u64 @24
struct DeviceInfoTopic (v1, 32 bytes)   deviceClass: u32 @0 (Mobile=0,
Desktop=1, Console=2, Media=3), dynamicPlatform: u32 @4 (AdaptiveMobile=0,
AdaptiveDesktop=1, Console=2, Media=3), formFactor: u32 @8 (Phone=0, Tablet=1,
PassportFold=2, BookFold=3, TriFold=4, Laptop=5, Desktop=6, Handheld=7, Tv=8),
certified: u8 @12, attested: u8 @13, reserved: [u8;18] @14
struct LimitsInfo (v1, 96 bytes)   maxHandles: u32 @0, maxChannelMessageBytes:
u32 @4, maxChannelHandles: u32 @8, maxChannelQueueMessages: u32 @12,
```

```
maxChannelQueueBytes: u64 @16, maxPortUserPackets: u32 @24, defaultStackSize:
u32 @28, maxPathBytes: u32 @32, maxNameBytes: u32 @36, maxFileSize: u64 @40,
maxMappingBytes: u64 @48, maxThreads: u32 @56, maxProcessesInJob: u32 @60,
maxStreamBuffers: u32 @64, maxLogMessageBytes: u32 @68, reserved: [u64;3] @72
struct SessionInfo (v1, 32 bytes)   appId: u64 @0, processId: u64 @8, jobId:
u64 @16, isMain: u8 @24, reserved: [u8;7] @25
```

Synchronous (shared read-only page mapped at init). `System.info` is the only place a consumer learns the page size, the backend kind, or the device class; it must never infer them.

---

21. Kernel backends

LibTessera is one portable front (everything above) plus one **backend** per kernel. The backend is the only code that knows a kernel exists: ~15 % of the library by size, replaceable as a unit. Its contract is the object semantics in §3–20; the tables below show each backend satisfies them.

21.1 Linux backend mapping

Object / call	Linux mechanism
Handle table	in-process table `{fd or pointer, type, rights, generation}`; fds are `O_CLOEXEC` and never exposed
Port	`io_uring` per port + eventfd side-queue for service completions; `Port.wait` = `io_uring_enter(GETEVENTS)` with absolute timeout
`Object.waitAsync`	`IORING_OP_POLL_ADD` (multishot for `Repeat`); `PEER_CLOSED` = `POLLHUP`
Event / EventPair	futex word + eventfd / `AF_UNIX` socketpair
Timer	`IORING_OP_TIMEOUT` (absolute, `CLOCK_MONOTONIC` / `BOOTTIME`), `timerfd` + `CLOCK_BOOTTIME_ALARM` for `Wake`
Channel	`socketpair(AF_UNIX, SOCK_SEQPACKET)`, `SCM_RIGHTS`, descriptor record per handle, `SO_PEERCRED` to trust rights from system peers
Memory	`memfd_create` (+ seals), `mmap` / `munmap` / `mprotect` / `madvise`, `udmabuf` / dma-buf for DeviceShareable
Thread	`clone3` with `CLONE_SETTLS`, `sched_setattr`, `futex2`, `tgkill` (internal signal, `Thread.kill` only)

Object / call	Linux mechanism
Process / Job	Launcher: `clone3(CLONE_INTO_CGROUP)` + `execveat`; `pidfd` for handles; cgroup v2 for jobs (`memory.max`, `pids.max`, `cpu.weight`, `cgroup.kill`, `cgroup.freeze` for suspend)
Sandbox	user + mount + pid + net namespaces (net ns only if no network entitlement), seccomp-bpf filter generated from the entitlement set, Landlock on container roots, no-new-privs
Namespace	service holds root fds; `openat2(RESOLVE_BENEATH|RESOLVE_NO_SYMLINKS)`, `getdents64`, `statx`, `renameat2`, `unlinkat`, `inotify` / `fanotify`; client sandbox has **no** path syscalls
File data	`IORING_OP_READV/WRITEV/FSYNC/FALLOCATE`, `ftruncate`, `flock` / `OFD` locks, `mmap` on the fd
Endpoint	`socket`, `bind`, `listen`, `setsockopt`, `IORING_OP_ACCEPT/CONNECT/SENDMSG/RECVMSG/SHUTDOWN`, `sendmmsg` / `recvmmsg`
Stream	Devices service: ALSA/V4L2/IIO/FF ioctls; buffers as dma-buf; per-stream channel
Gpu / Fence	DRM render node fd, dma-buf, `sync_file`, `drm_syncobj`; `Gpu.control` = ioctl pass-through for `tessera.driver`
Power	Power service publishes the shared page; wakeup sources, `power_supply`, `thermal_zone`
Random	`getrandom`
Time	vDSO `clock_gettime`
Log / Trace	shared rings to the logger/tracer services (no syscall on the hot path)
Entry / TLS	`_tessera_start` asm per arch; ELF `PT_TLS` template; `TPIDR_EL0` / `FS` set via `clone3` / `arch_prctl`

21.2 Linux app seccomp allowlist (generated; base set)

`read`, `write`, `readv`, `writev`, `pread64`, `pwrite64`, `close`, `fstat`, `statx` (fd-relative only, `AT_EMPTY_PATH`), `mmap`, `munmap`, `mprotect`, `madvise`, `mremap`, `memfd_create`, `ftruncate`, `fallocate`, `fsync`, `fdatasync`, `flock`, `fcntl` (OFD locks, `F_GETFL` only), `io_uring_setup/enter/register`, `eventfd2`, `timerfd_create/settime/gettime`, `clock_gettime`, `clock_getres`, `clock_nanosleep`, `nanosleep`, `futex`, `futex_waitv`, `futex_wake`, `futex_wait`, `futex_requeue`, `clone3` (thread flags only — `CLONE_THREAD` required, no `CLONE_NEWUSER`), `exit`, `exit_group`,

`sched_yield`, `sched_setattr`, `sched_getattr`, `set_tid_address`, `set_robust_list`, `rseq`, `getrandom`, `socketpair` (`AF_UNIX` only), `sendmsg`, `recvmsg` (`AF_UNIX`), `getsockopt(SO_PEERCRED)`, `shutdown`, `rt_sigprocmask`/`rt_sigaction`/`rt_sigreturn` (LibTessera's internal stub only, filtered by argument), `tgkill` (own tgid), `prctl(PR_SET_NAME, PR_SET_VMA)`, `getpid`, `gettid`, `pidfd_*` (own children), `ioctl` **denied**.

Added by entitlement: `tessera.network.*` → `socket` (`AF_INET/AF_INET6`, protocol by entitlement), `bind`, `listen`, `accept4`, `connect`, `setsockopt`, `sendmmsg`, `recvmmsg`, `getsockname`, `getpeername`; `tessera.jit` → `mprotect(PROT_EXEC)` with `PROT_WRITE`; `tessera.driver` → `ioctl` on the DRM/device fds; `tessera.thread.realtime` → `SCHED_DEADLINE`. Everything path-based (`open`, `openat`, `openat2`, `unlink*`, `rename*`, `mkdir*`, `getdents64`, `chdir`, `readlink*`, `execve*`, `mount`, …) is **never** in an app's filter.

21.3 XNU backend mapping

Object / call	XNU mechanism
Handle table	in-process table over Mach port names and fds
Port	`kqueue`; async I/O by an internal worker pool posting `EVFILT_USER`
`Object.waitAsync`	`EVFILT_READ/WRITE/MACHPORT/PROC/VNODE`
Event / EventPair	Mach ports + dead-name notifications
Timer	`EVFILT_TIMER` with `NOTE_MACHTIME`; wake via IOPM scheduled events
Channel	Mach messages with `MOVE_SEND` rights; port-destroyed notifications for `PEER_CLOSED`; handle rights are kernel-enforced (`BackendInfo.KERNEL_RIGHTS`)
Memory	`mach_make_memory_entry_64`, `mach_vm_map/protect/deallocate`, `vm_purgable_control`, IOSurface
Thread	`bsdthread_create`/`thread_create` + `thread_set_state`, `thread_policy_set`, `__ulock_wait2/wake`
Process / Job	Launcher `posix_spawn` + special ports for bootstrap; coalitions + `memorystatus` for jobs; `EVFILT_PROC` for termination
Sandbox	`sandbox_init` profile generated from entitlements
Namespace	service uses `openat` + `O_NOFOLLOW`/`fcntl(F_GETPATH)` containment checks, FSEvents/`EVFILT_VNODE`
Endpoint	BSD sockets, worker pool

Object / call	XNU mechanism
Gpu / Fence	IOAccelerator, IOSurface, `MTLSharedEvent`
Entry / TLS	`_tessera_start` per arch; Mach-O `__thread_vars`; `TPIDRRO_EL0` via `thread_set_tsd_base`

21.4 Enforcement ledger

Guarantee	Linux	XNU	Native kernel
App cannot name any path	kernel (seccomp: no path syscalls)	kernel (sandbox profile)	kernel (no namespace calls)
File Read/Write per handle	kernel (open flags on the fd)	kernel	kernel
List/Create/ Delete/Traverse per Node	Namespace service (client has no directory access at all)	Namespace service	kernel or service
Handle rights within a process (Duplicate/ Transfer/…)	LibTessera (advisory: a hostile process only limits itself; it cannot gain anything because the kernel object behind the handle already carries the real limit)	kernel (Mach rights)	kernel
Rights on handles received from the system	descriptor record trusted only from `SO_PEERCRED`-verified system peers	kernel	kernel
Entitlements (network, jit, realtime, devices)	kernel (seccomp/namespaces derived at launch) + services	kernel (sandbox) + services	kernel
Process/Job limits and kill	kernel (cgroup v2)	kernel (coalitions/ memorystatus)	kernel
No ambient authority	kernel (only fd 0 inherited; empty mount namespace outside containers)	kernel	kernel

Guarantee	Linux	XNU	Native kernel
Interaction causality	MosaicDisplay + Presence service (kernel-independent)	same	same

The Linux row for in-process handle rights is the one place enforcement is *advisory*; it is safe because every right that matters across a trust boundary (file access, network, devices, other processes) is backed by a kernel object or a service that re-checks. The `BackendInfo.KERNEL_RIGHTS` bit reports when a backend enforces handle rights in the kernel, for tooling.

21.5 The kernel-specific residue

Everything a port of LibTessera to a new kernel must write: the entry stub and syscall trampolines per architecture; the handle-table adapter; the twelve object implementations in §21.1; the sandbox generator; the executable-format TLS template reader; and the four service back-halves that touch hardware (Namespace, Devices, Power, Display). Nothing above the backend boundary — no struct, no call, no error, no protocol — changes.

---

22. Consumers

22.1 Tessera Rust std

The Rust std for Tessera is a `target_os = "tessera"` port whose `sys` layer links `libtessera` and nothing else. No libc, no `errno`, no `RawFd`.

Rust std	LibTessera
`alloc::System`	`Memory.reserve` + `Memory.map/commit/decommit` (an arena allocator over VM objects)
`thread::spawn`, `Builder`, `park/unpark`, `sleep`, `yield_now`	`Thread.spawn`, `Thread.sleep`, `Thread.yield`, `Sync.wait32/wake`
`sync::{Mutex, RwLock, Condvar, Once}`	futex implementations over `Sync.wait32/wake/requeue`

Rust std	LibTessera
`thread_local!`	ELF/Mach-O TLS initialised by `Thread.spawn` + `Thread.tlsBase`
`time::{Instant, SystemTime}`	`Time.read(Monotonic)`, `Time.read(Realtime)`
`fs::File::open/create`	`Node.lookup` / `Node.create` beneath a `Dir` handle, then `File.open`; `std::fs::Dir`-style API is the natural shape (`Path` is relative to a `Node`)
`Read/Write/Seek` on `File`	`File.read/write` with an explicit offset kept in the `File` struct
`fs::read_dir`, `metadata`, `remove_*`, `rename`, `create_dir`	`Node.list`, `Node.info`, `Node.delete`, `Node.rename`, `Node.create`
`fs::canonicalize`, absolute `Path`	**not available**; `Node.displayPath` for display only
`env::args`, `env::var`	`LaunchInfo.arguments`; there are no environment variables (`env::var` always `NotPresent`)
`env::current_dir`	`LaunchInfo.workingDirectoryHint` as a display string; no authority
`net::{TcpStream, TcpListener, UdpSocket}`	`Endpoint.*`; `ToSocketAddrs` via `tessera.resolver` (`Channel.call`)
`process::Command`	`Process.launch` with an executable `Node` from the bundle; `Stdio` = channels; `Child::wait` = `waitAsync(TERMINATED)` + `ProcessInfo.exitCode`
`process::exit`, `abort`	`Process.exit`; `abort` = `Process.exit(-1)` after `Log.write(Fault)`
`io::stdin/stdout/stderr`	the `StdIn/Out/Err` startup handles (Channel or File)
`os::tessera::*` (extension traits)	`AsHandle`, `FromHandle`, `Port`, `Completion`, `Rights`, grants and pickers — the async surface exposed raw for runtimes (Seyoul, Tokio-on-Tessera)
`rand` seed / `RandomState`	`Random.fill`
`panic` backtraces	`Trace` / `Log`, `Crash.setReporter` in debug

Blocking calls use the thread-port convention (§2.4). `std::os::tessera` is where the completion model surfaces for async runtimes; `std` itself stays synchronous.

22.2 A Tessera C std

`tessera.h` *is* the C system interface. A Tessera-native C standard library (distinct from the notarised POSIX-tier libc) would implement `stdio` / `stdlib` / `string` / `threads.h` over the same table as §22.1, with `FILE*` wrapping a `File` + offset and no `open(const char*)` — file handles come from a `Node`. This document does not design that library; it guarantees that every primitive it needs exists here.

22.3 The Kira runtime and frameworks

Kira's runtime on Tessera links LibTessera through the Rust std it re-exports (Kira app → Rust std → LibTessera → kernel). KiraUI/KCoreSurface reach MosaicDisplay through `tessera.display` channels; Kira Graphics' Tessera backend is a `tessera.driver` component using `Gpu.control`, `Gpu.allocate` and fences; Kira's crypto framework sits on `Random.fill`.

---

23. Call index

Sync unless marked **A** (asynchronous, `(port, key)` last). **143 calls.**

Handle / Object (9) `Handle.close`, `Handle.closeMany`, `Handle.duplicate`, `Handle.replace`, `Handle.info`, `Object.info`, `Object.setName`, `Object.waitAsync` **A**, `Object.signal` **Port (4)** `Port.create`, `Port.wait`, `Port.post`, `Port.cancel` **Memory (15)** `Memory.create`, `Memory.resize`, `Memory.slice`, `Memory.read`, `Memory.write`, `Memory.map`, `Memory.unmap`, `Memory.protect`, `Memory.commit`, `Memory.decommit`, `Memory.reserve`, `Memory.query`, `Memory.lock`, `Memory.unlock`, `Memory.setCachePolicy` **Thread / Sync (15)** `Thread.spawn`, `Thread.exit`, `Thread.current`, `Thread.id`, `Thread.setQos`, `Thread.setRealtime`, `Thread.sleep`, `Thread.yield`, `Thread.kill`, `Thread.tlsBase`, `Thread.setTlsBase`, `Sync.wait32`, `Sync.wait64`, `Sync.wake`, `Sync.requeue` **Time (7)** `Time.read`, `Time.resolution`, `Time.realtimeDetails`, `Timer.create`, `Timer.set` **A**, `Timer.setPeriodic` **A**, `Timer.cancel` **Event (2)** `Event.create`, `EventPair.create` **Channel (5)** `Channel.create`, `Channel.write`, `Channel.read`, `Channel.peek`, `Channel.call` **A** **Process / Job (9)** `Process.launch` **A**, `Process.exit`, `Process.current`, `Process.kill`, `Process.suspend`, `Process.resume`, `Job.create`, `Job.kill`,

`Job.setPolicy` **Namespace (11)** `Node.lookup` **A**, `Node.list` **A**, `Node.create` **A**, `Node.delete` **A**, `Node.rename` **A**, `Node.info` **A**, `Node.setAttributes` **A**, `Node.displayPath` **A**, `Node.watch`, `Watch.read`, `SharedContainer.open` **A File (11)** `File.open` **A**, `File.read` **A**, `File.write` **A**, `File.readMemory` **A**, `File.writeMemory` **A**, `File.sync` **A**, `File.resize` **A**, `File.allocate` **A**, `File.lock` **A**, `File.unlock`, `File.info` **Grants / Presence (8)** `Grant.request` **A**, `Picker.present` **A**, `Grant.restore` **A**, `Grant.release` **A**, `Grant.list` **A**, `Grant.info` **A**, `Authorization.request` **A**, `Settings.redirect` **A Network (14)** `Endpoint.create`, `Endpoint.bind`, `Endpoint.listen`, `Endpoint.accept` **A**, `Endpoint.connect` **A**, `Endpoint.send` **A**, `Endpoint.receive` **A**, `Endpoint.sendTo` **A**, `Endpoint.receiveFrom` **A**, `Endpoint.sendMany` **A**, `Endpoint.receiveMany` **A**, `Endpoint.shutdown`, `Endpoint.setOption`, `Endpoint.getOption` **Streams (9)** `Device.list` **A**, `Stream.open` **A**, `Stream.buffer`, `Stream.enqueue` **A**, `Stream.start` **A**, `Stream.stop` **A**, `Stream.flush` **A**, `Stream.reconfigure` **A**, `Stream.control` **A GPU / Fence (9)** `Gpu.list` **A**, `Gpu.open` **A**, `Gpu.allocate` **A**, `Gpu.import` **A**, `Gpu.control` **A**, `Fence.create`, `Fence.signal`, `Fence.waitAsync` **A**, `Fence.merge` **Power (3)** `Power.read`, `WakeLock.acquire` **A**, `Power.scheduleWake` **A Random (1)** `Random.fill` **Diagnostics (11)** `Log.write`, `Trace.enabled`, `Trace.begin`, `Trace.end`, `Trace.instant`, `Trace.counter`, `Trace.flow`, `Crash.setReporter`, `Debug.break`, `Debug.isAttached`, `System.info`

Plus the `tessera.system` protocol (§12), the wire format (§10.3) and the bootstrap message (§11.4), which are contracts rather than calls.

---

24. Decisions still open (for Priam)

1. **Sync-mode convenience.** This draft has no synchronous variants at all (thread-port convention). It is the cleanest, but every trivial `stat` from Rust std costs a `Port.wait`. Alternative: a `TESSERA_PORT_SYNC` sentinel that blocks in place. I recommend keeping it pure and measuring.
2. **Node info snapshot.** `Object.info(node, .Node)` returns a cached snapshot; is a possibly-stale sync path acceptable, or should Node info be async-only?
3. **Case sensitivity in user-visible locations.** Draft says case-sensitive/preserving with a service-level collision policy. Apple-style case-insensitive would be friendlier to users and worse for ports.
4. **Trash semantics.** `DeleteOptions.Permanent` defers to a per-volume policy the user controls. Confirm that Tessera wants a system trash at all (the immutable-system + container model doesn't require one).

5. **Job vs "Session".** I used `Job` (Zircon term). If Tessera wants its own word (`Realm`, `Cell`), it's a rename only.
6. **Camera as a Stream.** Camera *frames* fit the stream model; the camera *control* surface (`Stream.control` requests) will be large. If the settled camera scheme wants its own service protocol, `Stream` stays for frames only.
7. **`Gpu.import` returning the same handle.** Alternative is a distinct `GpuBuffer` object type. Same-handle keeps the "one Memory type" story; a distinct type is more explicit.
8. **Timeline fence on Linux without `drm_syncobj` support in a driver.** Fallback is service-emulated timelines; acceptable for non-GPU users, worth a `capabilities` bit.
9. **`Endpoint.sendMany` datagram struct** — 56 bytes with an inline `NetAddress`; could shrink to a pointer if QUIC stacks prefer their own layout.
10. **RISC-V.** The store mandate means `MachineInfo.arch` includes `RiscV64` and the entry stub/TLS/syscall residue exists for it; nothing else in this document is architecture-specific.